

Indice

Ideazione e analisi dei requisiti.....	3
Introduzione.....	3
Obiettivi e requisiti.....	3
Obiettivo dei casi d'uso.....	3
Casi d'uso.....	5
1. Cerca Itinerario.....	5
2. Timbra Biglietto.....	6
3. Monitora Automezzo.....	9
4. Gestisci Itinerari.....	10
5. Gestisci Automezzi.....	12
6. Visualizza Fermate.....	13
7. Gestisci Avvisi.....	14
8. Visualizza Avvisi.....	16
9. Invio Segnalazione.....	17
10. Visualizza Segnalazione.....	18
11. Gestisci Controllori.....	19
12. Inizio Corsa.....	20
13. Fine Corsa.....	22
14. Login Utente.....	23
15. Logout Utente.....	24
16. Registrazione Cliente.....	25
Documento di visione.....	26
Regole di business.....	26
Glossario.....	27
Analisi Orientata agli oggetti.....	28
Modello di dominio.....	28
Diagrammi di sequenza del sistema(SSD) e Contratti.....	29
Iterazione 1:.....	29
Iterazione 2:.....	31
Iterazione 3:.....	34
Iterazione 4:.....	40
Iterazione 5:.....	45
Progettazione.....	48
Diagramma delle classi.....	48
Diagrammi di interazione.....	49
Iterazione 1.....	49
UC1.....	49
Iterazione 2.....	51
UC2.....	51
UC3.....	53
Iterazione 3.....	53

UC4.....	53
UC5.....	55
UC6.....	57
UC9.....	57
UC10.....	58
Iterazione 4.....	59
UC2.....	59
UC4.....	60
UC5.....	61
UC7.....	63
UC8.....	64
UC11.....	64
UC12.....	66
UC13.....	66
Iterazione 5.....	67
UC3.....	67
UC15.....	68
UC16.....	68
Diagrammi di stato.....	69
Iterazione 2.....	69
Iterazione 4.....	69
Testing.....	70
Introduzione.....	70
Individuazione dei casi di test e testing unitario.....	70
Iterazione 1:.....	70
Iterazione 2:.....	72
Iterazione 3:.....	74
Iterazione 4:.....	77
Iterazione 5:.....	80
Test di sistema.....	82

Ideazione e analisi dei requisiti

Introduzione

Scopo e contesto: L'applicazione PickYourLine è stata sviluppata per offrire una soluzione software innovativa e intuitiva per il monitoraggio dei mezzi extraurbani, rispondendo alle esigenze sia degli utenti finali che degli operatori di trasporto extraurbano. Verrà garantita affidabilità e scalabilità, oltre che la compatibilità multiplatforma. L'obiettivo principale è fornire uno strumento efficace per tracciare in tempo reale la posizione dei mezzi, ottimizzare la gestione degli itinerari e migliorare l'accessibilità delle informazioni agli utenti.

Visione: l'applicazione PickYourLine si pone come supporto al sistema di gestione di trasporto extraurbano, garantendo una soluzione software per il monitoraggio dei servizi di trasporto attualmente attivi. Viene colmata la mancanza di un servizio digitale che garantisca informazioni in maniera intuitiva e versatile, agevolando l'esperienza utente e garantendo una soluzione utile e di facile utilizzo.

Obiettivi e requisiti

Gestione e monitoraggio automezzi: attività correlate alla gestione degli automezzi (inserimento, modifica, eliminazione, visualizzazione, cambio di stato); monitoraggio degli automezzi attualmente in transito.

Gestione e ricerca itinerari: attività correlate alla gestione degli itinerari (inserimento, modifica, eliminazione, visualizzazione); ricerca itinerario date città di partenza e città di destinazione.

Gestione segnalazioni e avvisi: attività correlate alla gestione di avvisi (inserimento, visualizzazione); attività correlate alla gestione di segnalazioni (inserimento, visualizzazione);

Gestione controllori: attività correlate alla gestione dei controllori (inserimento, eliminazione, visualizzazione).

Gestione corse e timbratura biglietti: operazioni di inizio e fine corsa; timbratura dei biglietti durante una corsa in una data fermata.

Gestione dei ruoli dell'Utente: gli utenti hanno associati dei ruoli che gli conferiscono differenti permessi all'interno dell'applicazione.

Obiettivo dei casi d'uso

Attore	Obiettivo	Caso d'uso
Utente	Definire un itinerario data posizione di partenza e posizione di destinazione	UC1: Cerca un Itinerario
Controllore	Inserire informazioni sui biglietti	UC2: Timbra biglietto
Utente	Visualizzare le informazioni relative alla posizione	UC3 Monitora un Automezzo

	dell'automezzo e relativa disponibilità di posti	
Amministratore	Operazioni CRUD sugli itinerari	UC4: Gestisci itinerari
Amministratore	Operazioni CRUD sugli automezzi	UC5: Gestisci automezzi
Utente	Visualizzare fermate data la città richiesta dall'utente	UC6: Visualizza fermate
Amministratore	Operazioni CRUD per gestire avvisi verso l'utente su ritardi o interruzioni del servizio	UC7: Gestisci avvisi
Utente	Visualizzare gli avvisi relativi al servizio	UC8: Visualizza avvisi
Cliente	Fornire informazioni su problematiche non previste dal servizio	UC9: Inserisci segnalazione
Amministratore	Visualizza le segnalazioni dell'Utente	UC10: Visualizza segnalazioni
Amministratore	Operazioni CRUD sui controllori.	UC11: Gestisci controllori
Controllore	Il Controllore sceglie l'automezzo e il relativo itinerario da supervisionare	UC12: Inizio corsa
Controllore	Il Controllore termina la supervisione dell'automezzo scelto	UC13: Fine corsa
Utente	Accedere al sistema per utilizzarne le funzioni in base al tipo di utente	UC14: Login Utente
Utente	Eseguire il logout dal sistema	UC15: Logout Utente
Cliente	Registrare un profilo Cliente per utilizzare il sistema come tale	UC16: Registrazione Cliente

Casi d'uso

1. Cerca Itinerario

Nome del caso d'uso	Cerca itinerario
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Utente
Parti interessate ed interessi	Utente: vuole cercare degli itinerari data posizione di partenza e posizione di destinazione
Pre-condizioni	L' applicazione è in possesso dei dati relativi al traffico
Garanzie di successo	L' Utente visualizza gli itinerari basati sulle posizioni da lui fornite
Scenario principale di successo	1. L' Utente vuole cercare un itinerario date posizioni di partenza e destinazione 2. L' Utente sceglie l'operazione "Cerca Itinerario" 3. Il Sistema fornisce una lista di città di partenza 4. L'Utente inserisce il codice della città di partenza 5. Il Sistema fornisce una lista di città di destinazione 6. L'Utente inserisce il codice della città di destinazione 7. Il Sistema sulla base dei dati immessi, fornisce le informazioni relative agli itinerari 8. L' Utente ha visualizzato il risultato 9. L' Utente sceglie un Itinerario da visualizzare in dettaglio 10. Il Sistema restituisce tutte le fermate relative all'Itinerario scelto dall'Utente I passi 9 e 10 vengono ripetuti finché serve 11. Il Sistema riporta l'Utente al Menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Utente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato

	4a: L' Utente inserisce un codice di città di partenza non valido 1. Il Sistema genera un messaggio di errore 2. L' Utente ripete il passo 4 inserendo un codice alternativo 6a: L' Utente inserisce un codice codice di città di arrivo non valido 1. Il Sistema genera un messaggio di errore 2. L' Utente ripete il passo 6 inserendo un codice alternativo 6b: L' Utente inserisce lo stesso codice di città in partenza e destinazione 1. Il Sistema genera un messaggio di errore 2. L'Utente ripete il passo 6 inserendo un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione molto frequente, legata al numero di utenze
Varie	

2. Timbra Biglietto

Nome del caso d'uso	Timbra biglietto
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Controllore
Parti interessate ed interessi	Cliente: vuole timbrare il biglietto, fornendo le relative informazioni al Controllore Controllore: inserisce nel Sistema le informazioni del biglietto
Pre-condizioni	Il Cliente possiede un biglietto L' Automezzo deve avere dei posti disponibili Il Controllore deve essere autenticato
Garanzie di successo	Il Controllore inserisce i dati del biglietto e viene aggiornata la disponibilità di posti e la

	posizione dell'automezzo
Scenario principale di successo	1. Il Controllore vuole timbrare il biglietto di un Utente 2. Il Controllore sceglie l'operazione "Timbra biglietto" 3. Il Controllore aggiorna la posizione attuale dell'automezzo inserendo il nome della fermata. 4. Il Sistema aumenta la disponibilità dei posti sulla base dei Clienti che scendono. 5. Il Controllore inserisce il codice del biglietto, codice della città di partenza e codice città di destinazione. 6. Il Controllore conferma l'inserimento e il Sistema registra il biglietto inserito 7. Il Sistema diminuisce la disponibilità dei posti sulla base dei biglietti inseriti I passi 5, 6, 7 vengono ripetuti finché serve 8. Il Controllore indica al Sistema di aver terminato l'operazione 9. Il Sistema riporta l'Utente al Menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. Il Controllore riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 2a: Il Controllore non sta supervisionando nessun Automezzo. 1. Il Sistema genera un messaggio d'errore 2. Il Controllore viene riportato al menù principale 3a: Il Controllore inserisce una posizione precedente alla posizione attuale 1. Il Sistema genera un messaggio d'errore. 2. Viene ripetuto il passo 3. 3b: Il Controllore inserisce una posizione uguale alla posizione attuale 1. Il Sistema genera un messaggio d'errore. 2. Viene ripetuto il passo 3. 3c: Il Controllore inserisce una posizione non presente nel percorso.

	1. Il Sistema genera un messaggio d'errore. 2. Viene ripetuto il passo 3. 3d: Il Controllore inserisce una posizione che coincide con la città di destinazione. 1. Il Sistema genera un messaggio d'errore. 2. Viene ripetuto il passo 3. 5a: L' Automezzo non ha posti disponibili 1. Il Sistema genera un avviso 2. Il Sistema termina l'operazione "Timbra biglietto" 5b: Il Controllore inserisce un codice di una città non presente nel servizio (destinazione e/o destinazione) 1. Il Sistema genera un messaggio d'errore. 2. Il Controllore inserisce un codice alternativo 5c: Il Controllore inserisce un biglietto con città di partenza diversa dalla posizione attuale. 1. Il Sistema genera un messaggio d'errore. 2. Viene ripetuto il passo 3. 5d: Il Controllore inserisce un biglietto con città di destinazione non presente nel percorso. 1. Il Sistema genera un messaggio d'errore. 2. Viene ripetuto il passo 3. 6a: Il Controllore non conferma l'inserimento 1. Il Sistema ritorna al passo 5 6b: Il Controllore conferma l'inserimento ma almeno un dato inserito non è corretto 1. Il Sistema genera un messaggio di errore 2. Il Sistema ritorna al passo 5
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione molto frequente, legata al numero di utenze

Varie	
-------	--

3. Monitora Automezzo

Nome del caso d'uso	Monitora Automezzo
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Utente
Parti interessate ed interessi	Cliente: richiede informazioni al sistema circa la posizione e stato di un automezzo
Pre-condizioni	L' applicazione è in possesso dei dati relativi agli automezzi
Garanzie di successo	Il Cliente visualizza le informazioni da lui richieste
Scenario principale di successo	1. Il Cliente vuole acquisire informazioni sulla posizione e stato di un automezzo 2. Il Cliente sceglie l'operazione "Monitora Automezzo" 3. Il Sistema fornisce una lista di automezzi in transito 4. Il Cliente inserisce il codice di un automezzo 5. Il Sistema restituisce le informazioni in dettaglio dell'automezzo scelto i passi 4 e 5 vengono ripetuti finché serve 6. Il Sistema riporta l'Utente al Menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. Il Controllore riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 4a: Il Cliente inserisce un codice relativo ad un automezzo non esistente 1. Il Sistema genera un messaggio di errore 2. Il cliente ripete il passo 4 inserendo un codice alternativo 4b: Il Cliente inserisci un codice relativo ad un automezzo non in transito 1. Il Sistema genera un messaggio di errore

	2. Il Cliente ripete il passo 4 inserendo un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione molto frequente, legata al numero di utenze
Varie	

4. Gestisci Itinerari

Nome del caso d'uso	Gestisci itinerari
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate ed interessi	Amministratore: inserisce, modifica, elimina, visualizza gli itinerari degli automezzi e i relativi orari di approdo alle varie fermate
Pre-condizioni	L'Amministratore deve essere autenticato
Garanzie di successo	Le operazioni di visualizzazione, modifica, inserimento e cancellazione di itinerari vengono correttamente eseguite
Scenario principale di successo	1. L'Amministratore vuole visualizzare/inserire/modificare/eliminare itinerari dal sistema 2. L'Amministratore sceglie l'operazione "Gestisci Itinerari" 3. L'amministratore sceglie un'operazione da eseguire tra "Visualizza elenco itinerari", "Inserimento Itinerario", "Modifica Itinerario" e "Elimina Itinerario" 4. Nel caso di inserimento e modifica vengono inseriti i dati relativi all'itinerario, mentre nel caso di cancellazione l'itinerario scelto viene eliminato dal sistema 5. Viene generato un avviso che notifica il risultato dell'operazione effettuata (a esclusione della

	visualizzazione) 6. L'Amministratore viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Amministratore riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 4a: L'Amministratore inserisce un codice non esistente (modifica e cancellazione) 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice alternativo 4b: L'Amministratore inserisce un codice già esistente (inserimento) 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice alternativo 4c: L'Amministratore inserisce degli orari non validi 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce degli orari alternativi 4d: L'Amministratore inserisce un percorso contenente meno di due città 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un percorso alternativo 4e: L'Amministratore inserisce un percorso contenente più volte la stessa città 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un percorso alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione poco frequente, legata a variazioni di traffico o condizioni stradali
Varie	

5. Gestisci Automezzi

Nome del caso d'uso	Gestisci automezzi
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate ed interessi	Amministratore: inserisce un nuovo automezzo, visualizza tutti gli automezzi, modifica o elimina gli automezzi attualmente non in transito
Pre-condizioni	L'Amministratore deve essere autenticato
Garanzie di successo	Le operazioni di modifica, inserimento e cancellazione di automezzi vengono correttamente eseguite
Scenario principale di successo	1. L'Amministratore vuole inserire/modificare/eliminare automezzi dal sistema 2. L'Amministratore sceglie l'operazione "Gestisci Automezzi" 3. L'amministratore sceglie un'operazione da eseguire tra "Visualizza elenco automezzi", "Inserimento Automezzo", "Modifica Automezzo" e "Elimina Automezzo" 4. Nel caso di inserimento e modifica vengono inseriti i dati relativi all'automezzo, mentre nel caso di cancellazione l'automezzo scelto viene eliminato dal sistema. 5. Viene generato un avviso che notifica il risultato dell'operazione effettuata (a esclusione della visualizzazione) 6. L'Amministratore viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Amministratore riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 4a: L'Amministratore inserisce un codice non esistente (modifica e cancellazione) 1. Viene visualizzato un messaggio di

	errore 2. L'Amministratore inserisce un codice alternativo 4b: L'Amministratore inserisce un codice già esistente (inserimento) 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice alternativo 4c: L'Amministratore inserisce il codice di un itinerario non esistente 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice itinerario alternativo 4d: Non ci sono automezzi non in transito(modifica) 1. Viene visualizzato un messaggio di errore 2. L' Amministratore viene riportato nel menù principale
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione poco frequente, legata a variazioni di traffico, condizioni stradali, segnalazioni
Varie	Le decisioni relative alla gestione degli automezzi possono essere influenzate dalle segnalazioni inviate dai Clienti

6. Visualizza Fermate

Nome del caso d'uso	Visualizza fermate
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Utente
Parti interessate ed interessi	Utente: visualizza tutte le fermate presenti nella città inserita
Pre-condizioni	Il sistema deve possedere le informazioni

	relative a città e fermate
Garanzie di successo	Il sistema restituisce le fermate relative alla città inserita
Scenario principale di successo	1. L'Utente vuole conoscere le fermate di una città. 2. L'Utente sceglie l'operazione "Visualizza Fermate" 3. Il sistema mostra le città in cui è attivo il servizio 4. L'Utente inserisce il codice di una città 5. Il sistema restituisce l'elenco delle fermate della città in questione I passi 3,4,5 vengono ripetuti fino a quando il Cliente lo ritiene necessario 6. L'Utente indica di aver terminato l'operazione e viene riportato al menù principale.
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Utente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 4a: L'Utente inserisce un codice non esistente 1. Viene visualizzato un messaggio di errore 2. L'Utente inserisce un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione frequente, legata al numero di utenze
Varie	

7. Gestisci Avvisi

Nome del caso d'uso	Gestisci avvisi
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Amministratore

Parti interessate ed interessi	Amministratore: inserisce, modifica, elimina avvisi che riguardano eventuali eventi imprevisti che possono influenzare il servizio
Pre-condizioni	L'Amministratore deve essere autenticato
Garanzie di successo	Le operazioni di modifica, inserimento e cancellazione di avvisi vengono correttamente eseguite
Scenario principale di successo	1. L'Amministratore vuole inserire/modificare/eliminare avvisi dal sistema 2. L'Amministratore sceglie l'operazione "Gestisci Avvisi" 3. L'amministratore sceglie un'operazione da eseguire tra "Inserimento Avviso", "Modifica Avviso" e "Elimina Avviso" 4. Nel caso di inserimento e modifica vengono inseriti i dati relativi all'avviso, mentre nel caso di cancellazione l'avviso scelto viene eliminato dal sistema. 5. L'Amministratore viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Utente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 4a: L'Amministratore inserisce un codice non esistente (modifica e cancellazione) 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Frequenza dell'operazione legata al numero di comunicazioni da effettuare
Varie	Le decisioni relative alla gestione degli automezzi possono essere influenzate dagli avvisi inviati dagli Amministratori

8. Visualizza Avvisi

Nome del caso d'uso	Visualizza avvisi
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Utente
Parti interessate ed interessi	Utente: visualizza gli avvisi che riguardano eventuali eventi imprevisti che possono influenzare il servizio
Pre-condizioni	-
Garanzie di successo	L'operazione di visualizzazione viene correttamente eseguita
Scenario principale di successo	1. L'Utente vuole visualizzare gli avvisi dal sistema 2. L'Utente sceglie l'operazione "Visualizza Avvisi" 3. Il sistema mostra l'elenco degli avvisi 4. L'Utente inserisce il codice dell'avviso che vuole approfondire 5. Il sistema mostra l'avviso in dettaglio I passi 3,4,5 vengono ripetuti finché l'Utente lo ritiene opportuno 6. L'Utente viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Utente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 3a: Non sono presenti avvisi nel sistema 1. Viene visualizzato un messaggio di errore 2. L'Utente viene riportato al menù principale 4a: L'Utente inserisce un codice non esistente 1. Viene visualizzato un messaggio di errore 2. L'Utente inserisce un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	

Frequenza di ripetizioni	Operazione frequente, legata al numero di utenze
Varie	

9. Invio Segnalazione

Nome del caso d'uso	Invio segnalazione
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Cliente
Parti interessate ed interessi	Cliente: vuole inviare una segnalazione agli amministratori del servizio
Pre-condizioni	Il Cliente deve essere autenticato
Garanzie di successo	La segnalazione è stata correttamente inviata
Scenario principale di successo	1. Il Cliente vuole scrivere una segnalazione 2. Il Cliente sceglie l'operazione "Invio Segnalazione" 3. Il Cliente inserisce l'oggetto e il contenuto della segnalazione 4. Il Cliente conferma l'invio 5. Il Cliente viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. Il Cliente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione poco frequente, legata al numero di imprevisti
Varie	

10. Visualizza Segnalazione

Nome del caso d'uso	Visualizza segnalazioni
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate ed interessi	Amministratore: vuole visualizzare le segnalazioni ricevute dai clienti
Pre-condizioni	L'Amministratore deve essere autenticato
Garanzie di successo	Il sistema restituisce l'elenco delle segnalazioni
Scenario principale di successo	1. L'Amministratore vuole visualizzare le segnalazioni ricevute 2. L'Amministratore sceglie l'operazione "Visualizza Segnalazioni" 3. Il sistema mostra l'elenco delle segnalazioni 4. L'Amministratore inserisce il codice della segnalazione che vuole approfondire 5. Il sistema mostra la segnalazione in dettaglio I passi 3,4,5 vengono ripetuti fin quando l'Amministratore lo ritiene necessario 6. L'Amministratore indica di aver terminato l'operazione e viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Amministratore riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 3a: Non sono presenti segnalazioni nel sistema 1. Viene visualizzato un messaggio di avviso 2. L'amministratore viene riportato al menù principale 4a: L'Amministratore inserisce un codice non esistente 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice

	alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione frequente
Varie	

11. Gestisci Controllori

Nome del caso d'uso	Gestisci controllori
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Amministratore
Parti interessate ed interessi	Amministratore: inserisce un nuovo controllore, visualizza tutti i controllori o elimina gli automezzi attualmente non in transito
Pre-condizioni	L'Amministratore deve essere autenticato
Garanzie di successo	Le operazioni di visualizzazione, inserimento e cancellazione dei controllori vengono correttamente eseguite
Scenario principale di successo	1. L'Amministratore vuole inserire/visualizzare/eliminare controllori dal sistema 2. L'Amministratore sceglie l'operazione "Gestisci Controllori" 3. L'amministratore sceglie un'operazione da eseguire tra "Visualizza elenco controllori", "Inserimento Controllore" ed "Elimina Controllore" 4. Nel caso di inserimento vengono inseriti i dati relativi al controllore, mentre nel caso di cancellazione il controllore scelto viene eliminato dal sistema. 5. L'Amministratore viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Amministratore riavvia il Sistema e

	richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 4a: L'Amministratore inserisce un codice non esistente (cancellazione) 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice alternativo 4b: L'Amministratore inserisce un codice di un controllore che è in servizio (cancellazione) 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice alternativo 4c: L'Amministratore inserisce un codice già esistente (inserimento) 1. Viene visualizzato un messaggio di errore 2. L'Amministratore inserisce un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione poco frequente
Varie	

12. Inizio Corsa

Nome del caso d'uso	Inizio corsa
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Controllore
Parti interessate ed interessi	Controllore: inserisce l'automezzo che deve supervisionare
Pre-condizioni	Il Controllore deve essere autenticato L'automezzo scelto dal controllore deve essere in stato NonInTransito L'automezzo deve avere un itinerario

	assegnato
Garanzie di successo	L'automezzo viene associato al Controllore e cambia il suo stato in InTransito
Scenario principale di successo	1. Il Controllore sale sull'automezzo 2. Il Controllore sceglie l'operazione "Inizio Corsa" 3. Il sistema visualizza l'elenco degli automezzi in stato NonInTransito 4. Il Controllore inserisce il codice dell'automezzo che deve supervisionare 5. Il Controllore viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. Il Controllore riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 3a: Non ci sono automezzi non in transito 1. Viene visualizzato un messaggio di errore 2. Il Controllore viene riportato nel menù principale 3b: Il Controllore sta già supervisionando un automezzo 1. Viene visualizzato un messaggio di errore 2. Il Controllore viene riportato al menù principale 4a: Il Controllore inserisce un codice non esistente o non in transito 1. Viene visualizzato un messaggio di errore 2. Il Controllore inserisce un codice alternativo 4b: Il Controllore inserisce un codice di un automezzo privo di itinerario 1. Viene visualizzato un messaggio di errore 2. Il Controllore inserisce un codice alternativo
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione molto frequente, legata al

	numero di corse effettuate
Varie	

13. Fine Corsa

Nome del caso d'uso	Fine corsa
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Controllore
Parti interessate ed interessi	Controllore: vuole indicare la fine della supervisione dell'automezzo causa fine corsa
Pre-condizioni	Il Controllore deve essere autenticato Il Controllore deve avere un mezzo supervisionato associato
Garanzie di successo	L'automezzo viene dissociato dal Controllore e cambia il suo stato in NonInTransito
Scenario principale di successo	1. Il Controllore arriva a destinazione 2. Il Controllore sceglie l'operazione "Fine Corsa" 3. Il Controllore viene riportato al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. Il Controllore riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 2a: Il Controllore non supervisiona un automezzo 1. Viene visualizzato un messaggio di errore 2. Il Controllore viene riportato al menù principale
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione molto frequente, legata al numero di corse effettuate

Varie	
-------	--

14. Login Utente

Nome del caso d'uso	Login Utente
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Utente
Parti interessate ed interessi	Utente: vuole effettuare l'accesso all'applicazione
Pre-condizioni	L' applicazione è in possesso dei dati relativi all'utente
Garanzie di successo	L' Utente effettua l'accesso
Scenario principale di successo	1. L'Utente avvia il software 2. L'Utente sceglie l'operazione "Login Utente" 3. L'Utente inserisce codice e password 4. Il Sistema riporta l'Utente al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Utente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 3a: L'Utente è autenticato 1. Il Sistema genera un messaggio di errore 2. L'Utente viene riportato al menù principale 3b: L' Utente inserisce delle credenziali non valide 1. Il Sistema genera un messaggio di errore 2. L' Utente ripete il passo 2 inserendo credenziali alternative 3c: L' Utente inserisce un codice non esistente nel sistema 1. Il Sistema genera un messaggio di errore 2. L' Utente ripete il passo 2 inserendo credenziali alternative

Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione frequente, legata al numero di utenze
Varie	

15. Logout Utente

Nome del caso d'uso	Logout Utente
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Utente
Parti interessate ed interessi	Utente: vuole effettuare una disconnessione
Pre-condizioni	L' Utente deve essere autenticato
Garanzie di successo	L' Utente effettua la disconnessione
Scenario principale di successo	1. L'Utente vuole disconnettersi dall'applicazione 2. L'Utente sceglie l'operazione "Logout Utente" 3. Il Sistema riporta l'Utente al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. L'Utente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 2a: L' Utente non è autenticato 1. Il Sistema genera un messaggio di errore 2. L' Utente viene riportato al menù principale 2b: L'Utente è un Controllore e sta supervisionando un Automezzo 1. Il Sistema genera un messaggio di errore 2. L' Utente viene riportato al menù principale
Requisiti speciali	

Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione frequente, legata al numero di utenze
Varie	

16. Registrazione Cliente

Nome del caso d'uso	Registrazione Cliente
Portata	Applicazione PickYourLine
Livello	Obiettivo Utente
Attore primario	Cliente
Parti interessate ed interessi	Cliente: vuole effettuare la registrazione
Pre-condizioni	-
Garanzie di successo	Il Cliente viene registrato
Scenario principale di successo	1. Il Cliente vuole registrarsi nell'applicazione 2. Il Cliente sceglie l'operazione "Registrazione Cliente" 3. Il Cliente inserisce codice, password, nome e cognome 4. Il Sistema riporta l'Utente al menù principale
Estensioni	*a: Il Sistema si arresta improvvisamente 1. Il Cliente riavvia il Sistema e richiede il ripristino dello stato precedente 2. Il Sistema ripristina lo stato 3a: Il Cliente è autenticato 1. Il Sistema genera un messaggio di errore 2. Il Cliente viene riportato al menù principale 3b: Il Cliente inserisce un codice già esistente 1. Il Sistema genera un messaggio di errore 2. Il Cliente inserisce un codice alternativo 3c: Il Cliente inserisce un codice non valido 1. Il Sistema genera un messaggio di

	errore 2. Il Cliente inserisce un codice alternativo 3d: Il Cliente inserisce una password non valida 1. Il Sistema genera un messaggio di errore 2. Il Cliente inserisce una password alternativa
Requisiti speciali	
Elenco delle varianti tecnologiche e dei dati	
Frequenza di ripetizioni	Operazione frequente, legata al numero di nuove utenze
Varie	

Documento di visione

L'applicazione PickYourLine intende offrire una soluzione software innovativa e intuitiva per il monitoraggio dei mezzi extraurbani, rispondendo alle esigenze sia degli utenti finali che degli operatori di trasporto extraurbano.

Per ulteriori approfondimenti visionare il documento di visione in dettaglio.

Regole di business

ID	Regola	Modificabilità	Sorgente
R1	Per ogni città, in un itinerario, vengono attraversate tutte le fermate di quella città	bassa	Politica interna della compagnia di trasporto
R2	Per il calcolo della posizione di un automezzo, viene considerata la fermata dell'ultimo utente a cui è stato timbrato il biglietto	bassa	Politica interna della compagnia di trasporto
R3	Se la posizione di un automezzo in	bassa	Politica interna della compagnia di

	transito coincide con l'ultima città dell'itinerario percorso, allora non è possibile timbrare altri biglietti		trasporto
R4	Quando un automezzo subisce delle modifiche che impattano sul servizio, devono essere inviati degli avvisi all'utenza	bassa	Politica interna della compagnia di trasporto

Glossario

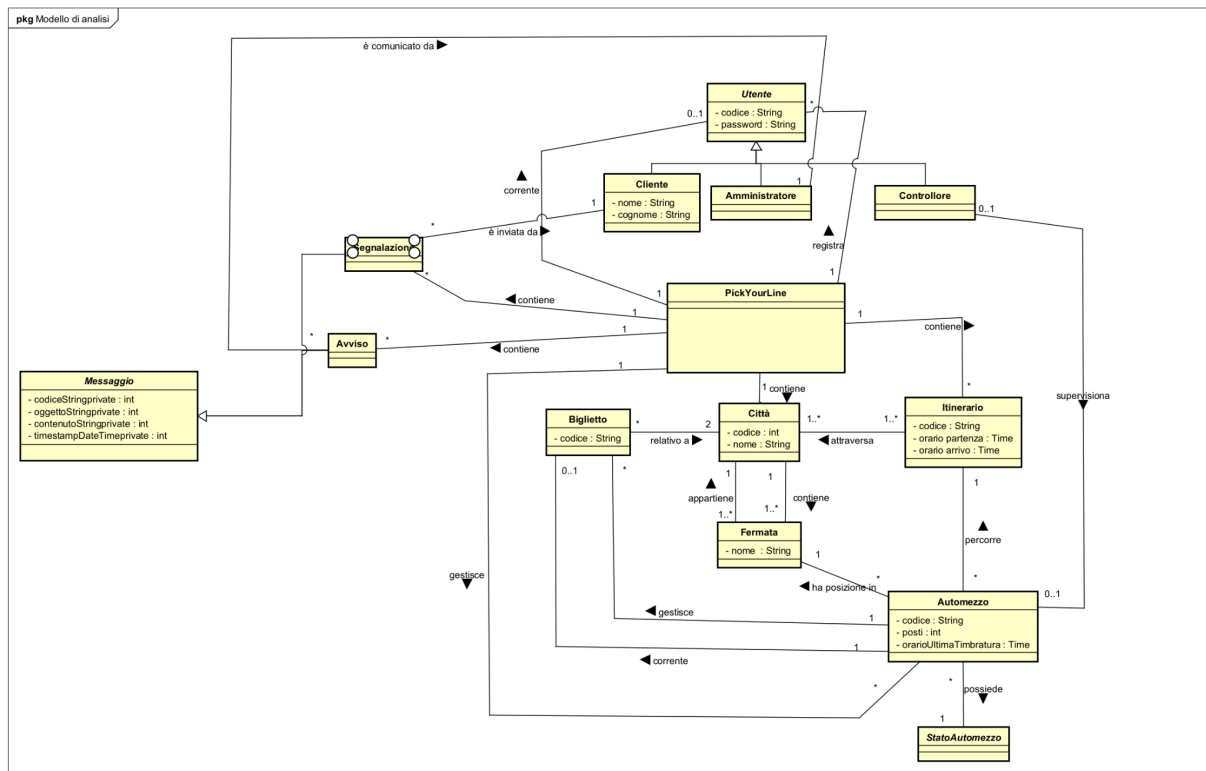
Termine	Definizione
Utente	Generico utente che utilizza il software
Controllore	Addetto alla supervisione dell'Automezzo e alla timbratura dei Biglietti
Amministratore	Gestore del servizio
Cliente	Cliente del servizio
Itinerario	Percorso tracciato da una posizione di partenza sino a una posizione di destinazione: incluse le posizioni intermedie
Corsa	Istanza di un Itinerario
Segnalazione	Messaggio inviato dall'Utente verso l'Amministratore, riguardante un problema non noto al Sistema
Avviso	Messaggio inviato dall'Amministratore verso gli Utente mediante apposito spazio
Fermata	Posizione di partenza o di arrivo, possono coincidere con nomi di stazioni, piazze o vie

Analisi Orientata agli oggetti

L'applicazione è stata sviluppata seguendo un approccio iterativo ed evolutivo suddiviso in cinque fasi. Ogni iterazione prevede la gestione dei rischi principali e un'analisi incrementale dei requisiti, riducendo al minimo gli effetti di possibili errori di progettazione e implementazione.

Modello di dominio

In questa fase è stato definito il modello di dominio, identificando tutti gli attori e le classi essenziali per la realizzazione dell'applicazione.



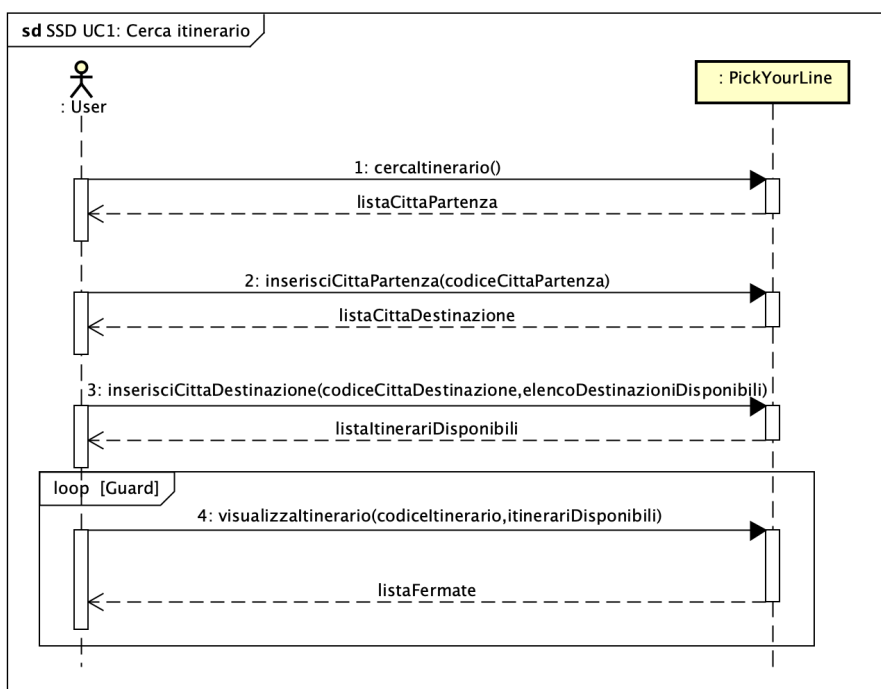
- **PickYourLine**: Rappresenta l'applicazione stessa responsabile di gestire tutte le richieste dei vari utenti che possono interagire con PickYourLine.
- **Automezzo**: Rappresenta un'entità che è in grado di percorrere un itinerario.
- **StatoAutomezzo**: Rappresenta i vari stati in cui si può trovare un automezzo.
- **Itinerario**: Rappresenta un percorso che viene percorso da un automezzo.
- **Città**: entità che rappresenta una città vera e propria che fa parte dell'itinerario.
- **Fermata**: Rappresenta un'entità fisica che esprime il concetto di fermata di un automezzo.
- **Utente**: Rappresenta i diversi tipi di utenti che la piattaforma può ospitare.

- **Cliente:** Rappresenta il cliente, il quale potrebbe possedere un biglietto.
- **Controllore:** Rappresenta un dipendente della compagnia che si occupa di supervisionare un automezzo durante una corsa e di timbrare dei biglietti durante il proprio turno lavorativo.
- **Amministratore:** Rappresenta un dipendente della compagnia che si occupa di gestire automezzi, segnalazioni e avvisi.
- **Biglietto:** Rappresenta l'entità fisica di un biglietto che un cliente possiede e che il controllore dovrà timbrare.
- **Messaggio:** Rappresenta i diversi tipi di messaggi che il sistema permette di inviare agli utenti.
- **Segnalazione:** Rappresenta un messaggio che viene inviato dal cliente all'amministratore per informare di eventuali problemi riguardo al servizio.
- **Avviso:** Rappresenta un messaggio che viene inviato dall'amministratore al cliente contenente informazioni sul servizio stesso.

Diagrammi di sequenza del sistema(SSD) e Contratti

Il passo successivo consiste nella realizzazione dei diagrammi di sequenza del sistema (SSD). In seguito, le operazioni di sistema identificate nei diagrammi verranno descritte in dettaglio mediante i Contratti, quest' ultimi formalizzano le responsabilità di ciascuna operazione, specificando condizioni pre e post-operazione, oltre alle eventuali eccezioni associate.

Iterazione 1:



Contratti UC1:

Operazione	cercaltinerario()
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	Esiste la lista delle città e dei relativi itinerari
Post-condizioni	Viene recuperata la lista contenente le istanze delle Città di partenza

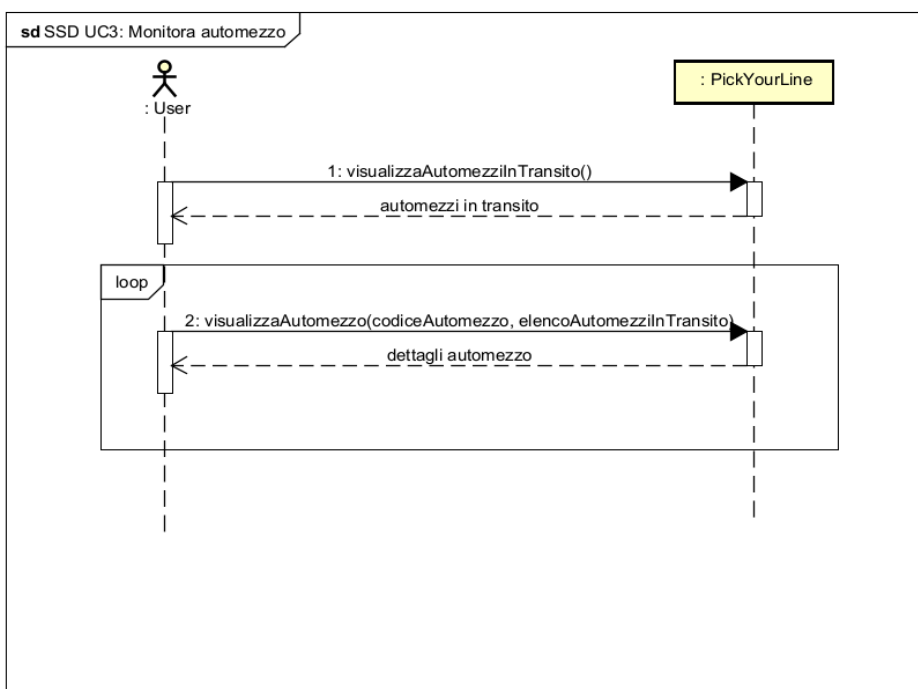
Operazione	inserisciCittaPartenza(codiceCittaPartenza)
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	È in corso la ricerca dell'itinerario
Post-condizioni	-Viene recuperata l'istanza della città di partenza cp scelta dall'utente -Viene associata l'istanza cp all'istanza di PickYourLine -Viene recuperata la lista contenente le istanze delle Città di destinazione

Operazione	inserisciCittaDestinazione(codiceCittaDestinazione)
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	È in corso la ricerca dell'itinerario
Post-condizioni	Viene recuperata la lista contenente le istanze di Itinerario

Operazione	visualizzaltinerario(codiceltinerario)
Riferimenti	UC1: Cerca itinerario
Pre-condizioni	È in corso la ricerca dell'itinerario

Post-condizioni	Viene recuperata la lista contenente le istanze di Fermate, relativa all' Itinerario scelto
-----------------	---

Iterazione 2:



Contratti UC2:

Operazione	aggiornaPosizioneAutomezzo(nomeFermata)
Riferimenti	UC2: Timbra biglietto
Pre-condizioni	- L'automezzo deve essere in transito
Post-condizioni	- Viene recuperata l'istanza di fermata sulla base di nomeFermata - Viene aggiornata la posizione dell'Automezzo -Vengono rimosse le istanze di biglietto da elencoBiglietti sulla base della posizione aggiornata

Operazione	timbraBiglietto(codiceBiglietto,codiceCittaPartenza,codiceCittaDestinazione)
Riferimenti	UC2: Timbra biglietto
Pre-condizioni	- Devono esserci dei posti disponibili nell'automezzo -Il controllore deve essere già associato all'automezzo tramite l'associazione supervisiona
Post-condizioni	- Viene recuperata l'istanza della città di partenza sulla base di codiceCittaPartenza - Viene recuperata l'istanza della città di destinazione sulla base di codiceCittaDestinazione - Viene creata un'istanza b di Biglietto - Vengono settati gli attributi di b - b viene associata a cittaPartenza tramite l'associazione relativo a -b viene associata a cittaDestinazione tramite l'associazione relativo a - b viene associata all' automezzo corrente mediante l'associazione corrente

Operazione	confermaInserimento()
Riferimenti	UC2: Timbra biglietto
Pre-condizioni	È in corso l'inserimento del biglietto
Post-condizioni	- b viene aggiunto in elencoBiglietti dell'automezzo corrente - l'associazione corrente tra b e l'automezzo corrente viene spezzata

Operazione	terminaInserimento
Riferimenti	UC2: Timbra biglietto
Pre-condizioni	È in corso l'inserimento del biglietto
Post-condizioni	Viene settato l'attributo orarioUltimaTimbratura dell'automezzo corrente

Contratti UC3:

Operazione	visualizzaAutomezziInTransito()
Riferimenti	UC3: Monitora automezzo
Pre-condizioni	Deve esistere una lista di itinerari a cui sono associati i relativi automezzi
Post-condizioni	Viene recuperata la lista contenente le istanze degli automezzi attualmente in transito

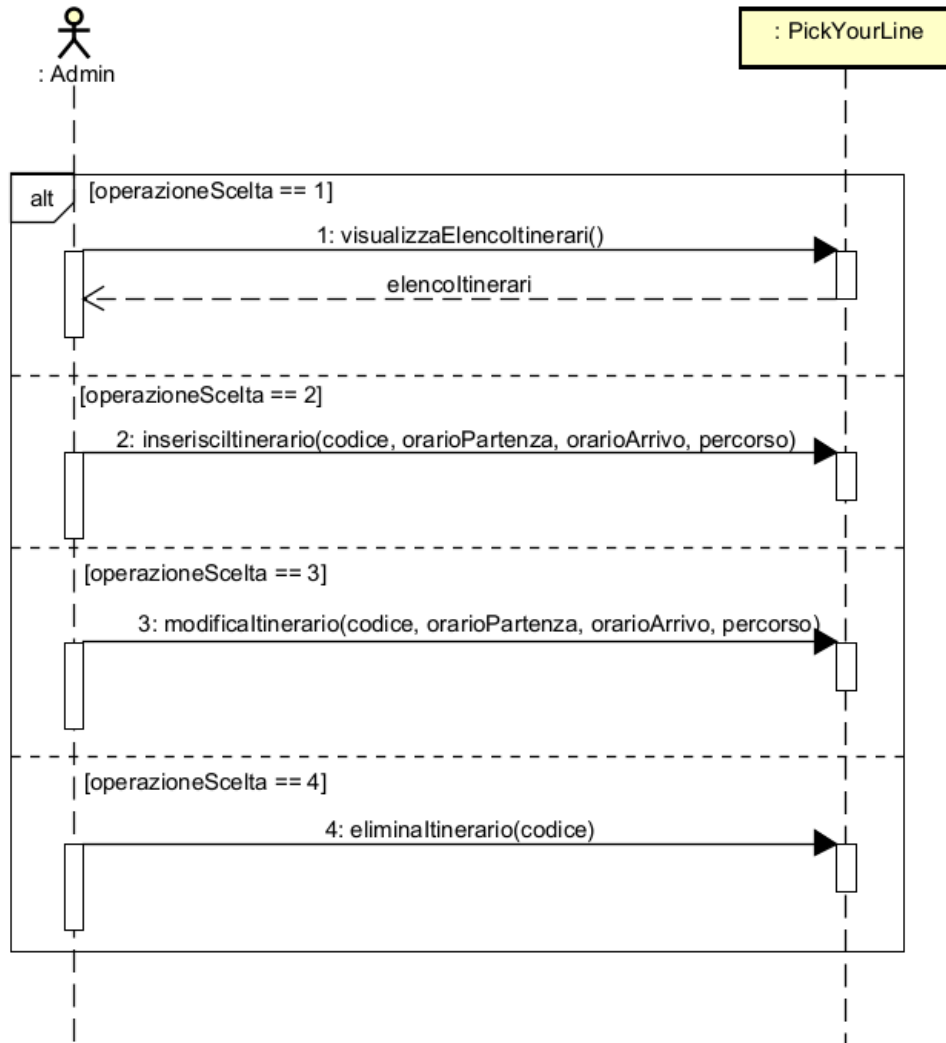
Operazione	visualizzaAutomezzo(codiceAutomezzo, elencoAutomezziInTransito)
Riferimenti	UC3: Monitora automezzo
Pre-condizioni	È in corso il monitoraggio degli automezzi attualmente in transito

Post-condizioni

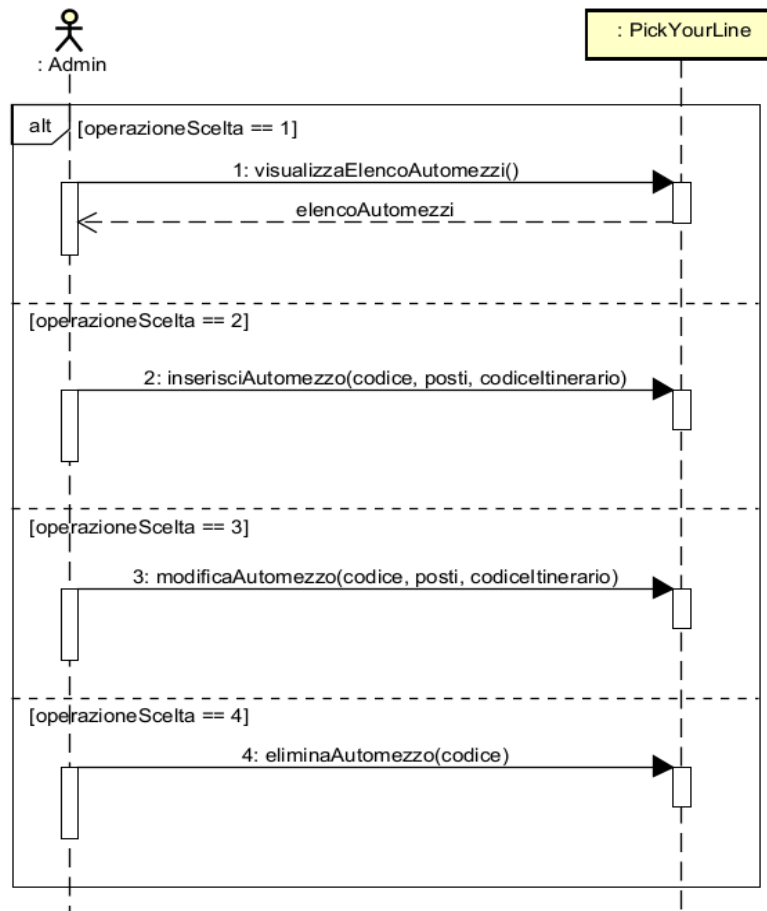
Viene recuperata l'istanza a di Automezzo
sulla base di codiceAutomezzo

Iterazione 3:

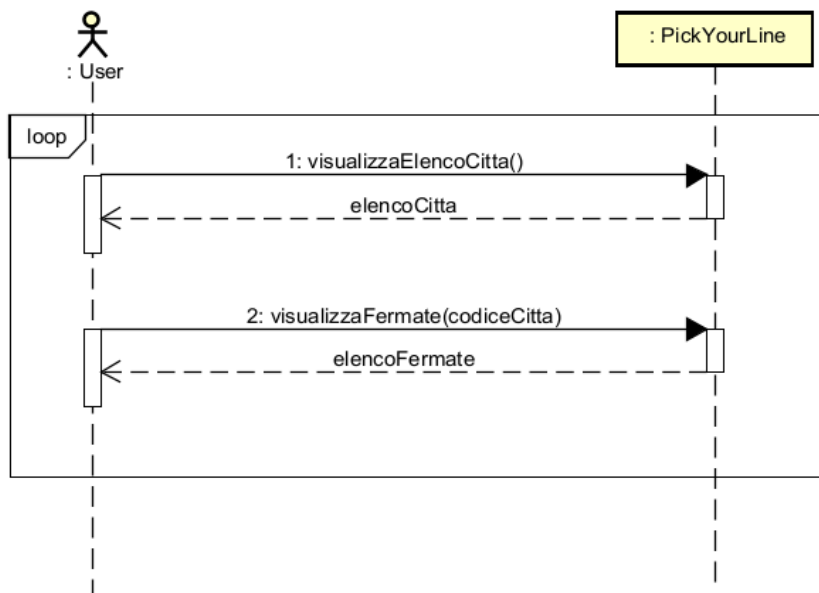
sd SSD UC4: Gestisci itinerari



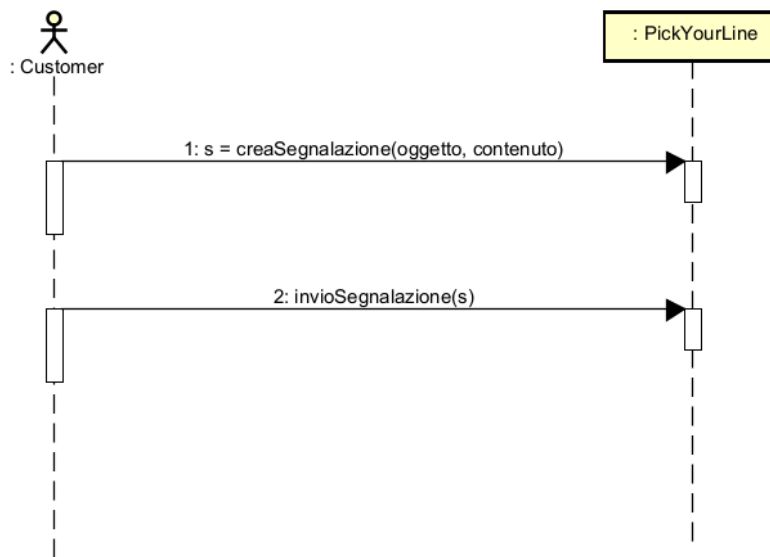
sd SSD UC5: Gestisci automezzi



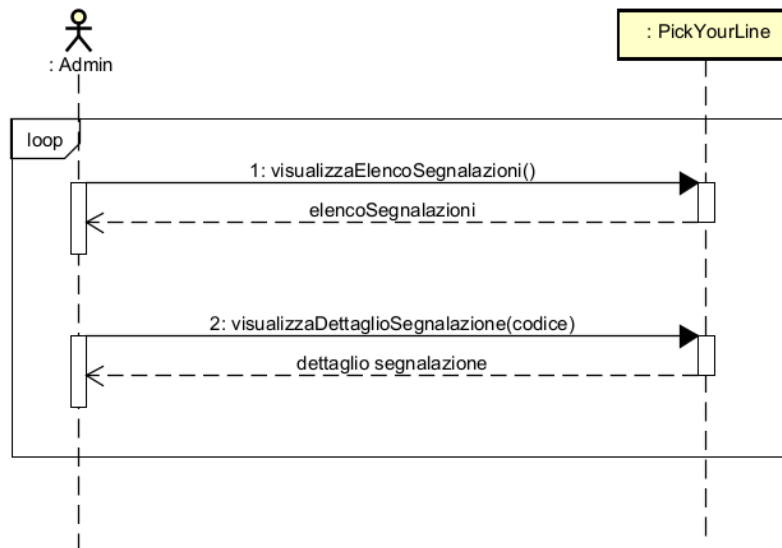
sd SSD UC6: Visualizza Fermate



sd SSD UC9: Invio segnalazione



sd SSD UC10: Visualizza segnalazioni



Contratti UC4:

Operazione	visualizzaElencoItinerari()
Riferimenti	UC4: Gestisci itinerari
Pre-condizioni	-
Post-condizioni	Viene recuperato l'elenco delle istanze di Itinerario

Operazione	inseriscItinerario(codice, orarioPartenza, orarioArrivo, percorso)
Riferimenti	UC4: Gestisci itinerari
Pre-condizioni	È in corso la gestione degli itinerari
Post-condizioni	- viene creata un'istanza i di Itinerario - i viene aggiunta all'elenco degli itinerari

Operazione	modificaltinerario(codice, orarioPartenza, orarioArrivo, percorso)
Riferimenti	UC4: Gestisci itinerari
Pre-condizioni	È in corso la gestione degli itinerari
Post-condizioni	- viene recuperata l'istanza i di Itinerario - vengono aggiornati gli attributi di i

Operazione	eliminaItinerario(codice)
Riferimenti	UC4: Gestisci itinerari
Pre-condizioni	È in corso la gestione degli itinerari
Post-condizioni	- viene recuperata l'istanza i di Itinerario - i viene eliminata dall'elenco degli itinerari

Contratti UC5:

Operazione	visualizzaElencoAutomezzi()
Riferimenti	UC5: Gestisci automezzi
Pre-condizioni	-
Post-condizioni	Viene recuperato l'elenco delle istanze di Automezzo

Operazione	inserisciAutomezzo(codice, posti, itinerario)
Riferimenti	UC5: Gestisci automezzi
Pre-condizioni	È in corso la gestione degli automezzi
Post-condizioni	- viene creata un'istanza a di Automezzo - a viene aggiunta all'elenco degli automezzi

Operazione	modificaAutomezzo(codice, itinerario)
Riferimenti	UC5: Gestisci automezzi
Pre-condizioni	È in corso la gestione degli automezzi
Post-condizioni	- viene recuperata l'istanza a di Automezzo - vengono aggiornati gli attributi di a

Operazione	eliminaAutomezzo(codice)
Riferimenti	UC5: Gestisci automezzi
Pre-condizioni	È in corso la gestione degli automezzi
Post-condizioni	- viene recuperata l'istanza a di Automezzo - a viene eliminata dall'elenco degli automezzi

Contratti UC6:

Operazione	visualizzaElencoCitta()
Riferimenti	UC6: Visualizza fermate
Pre-condizioni	Esiste l'elenco delle città e le delle relative fermate
Post-condizioni	Viene recuperato l'elenco delle istanze di Citta

Operazione	visualizzaFermate(codiceCitta)
Riferimenti	UC6: Visualizza fermate
Pre-condizioni	È in corso la visualizzazione delle fermate di una città
Post-condizioni	- viene recuperata l'istanza c di Citta sulla base di codiceCitta - viene recuperato l'elenco delle istanze di Fermata associato a c

Contratti UC9:

Operazione	creaSegnalazione(oggetto, contenuto)
Riferimenti	UC9: Invio segnalazione
Pre-condizioni	-
Post-condizioni	Viene creata un'istanza s di Segnalazione

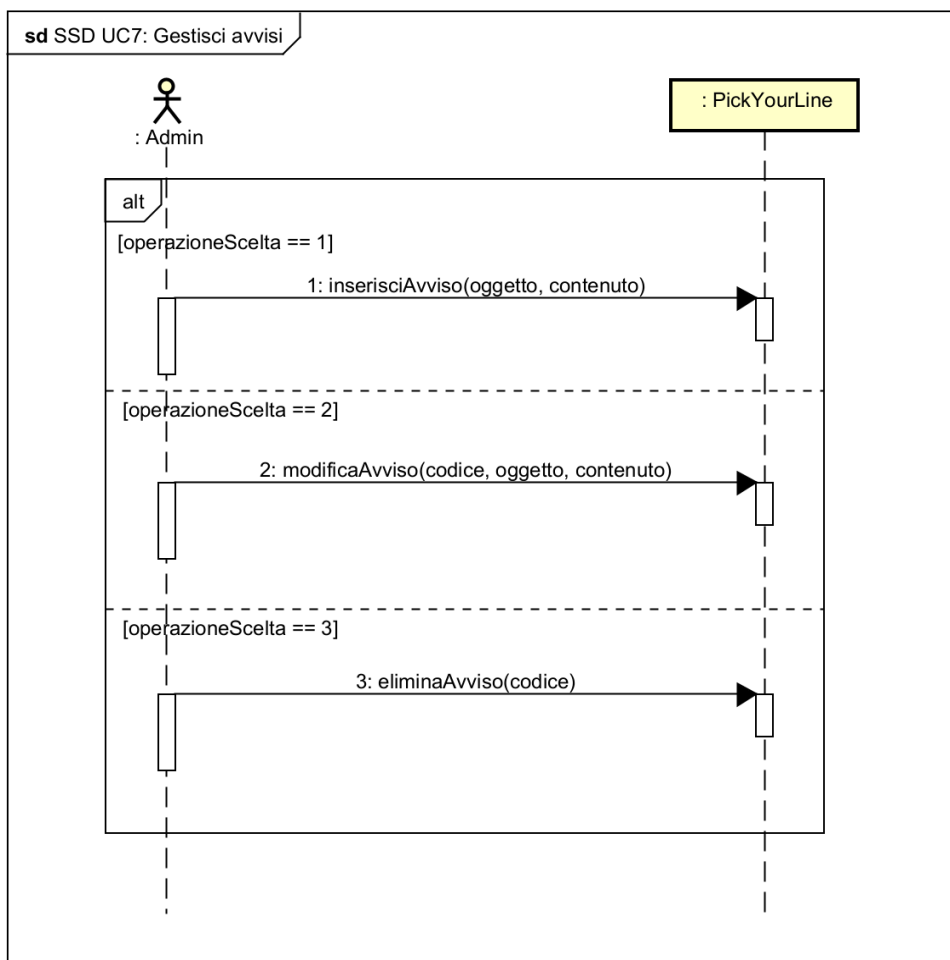
Operazione	invioSegnalazione(s)
Riferimenti	UC9: Invio segnalazione
Pre-condizioni	È in corso l'invio di una segnalazione
Post-condizioni	s viene aggiunta all'elenco delle segnalazioni

Contratti UC10:

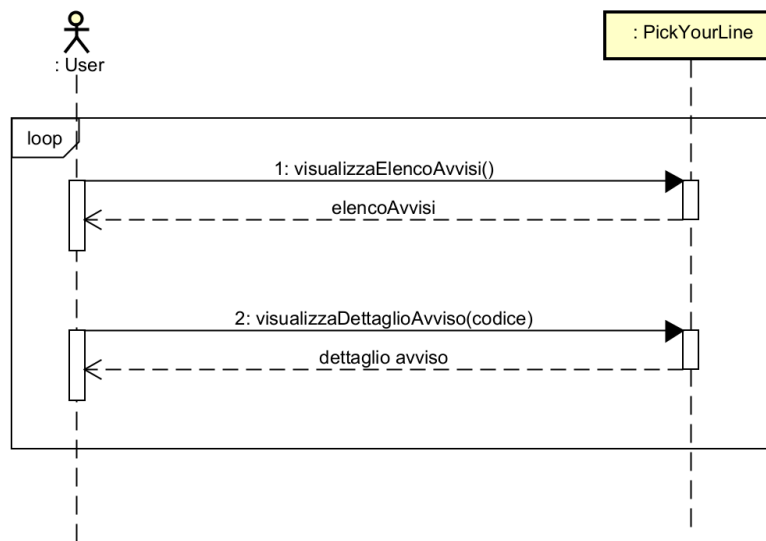
Operazione	visualizzaElencoSegnalazioni()
Riferimenti	UC10: Visualizza segnalazioni
Pre-condizioni	-
Post-condizioni	Viene recuperato l'elenco delle istanze di Segnalazione

Operazione	visualizzaDettaglioSegnalazione(codice)
Riferimenti	UC10: Visualizza segnalazioni
Pre-condizioni	È in corso la visualizzazione delle segnalazioni
Post-condizioni	viene recuperata l'istanza s di segnalazione sulla base del codice

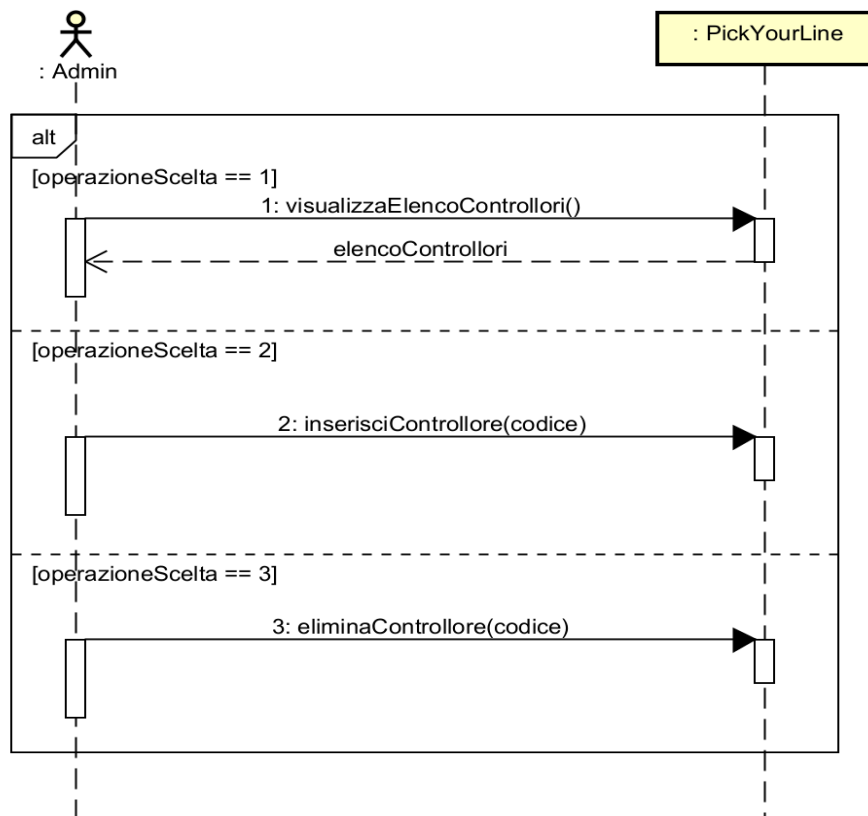
Iterazione 4:

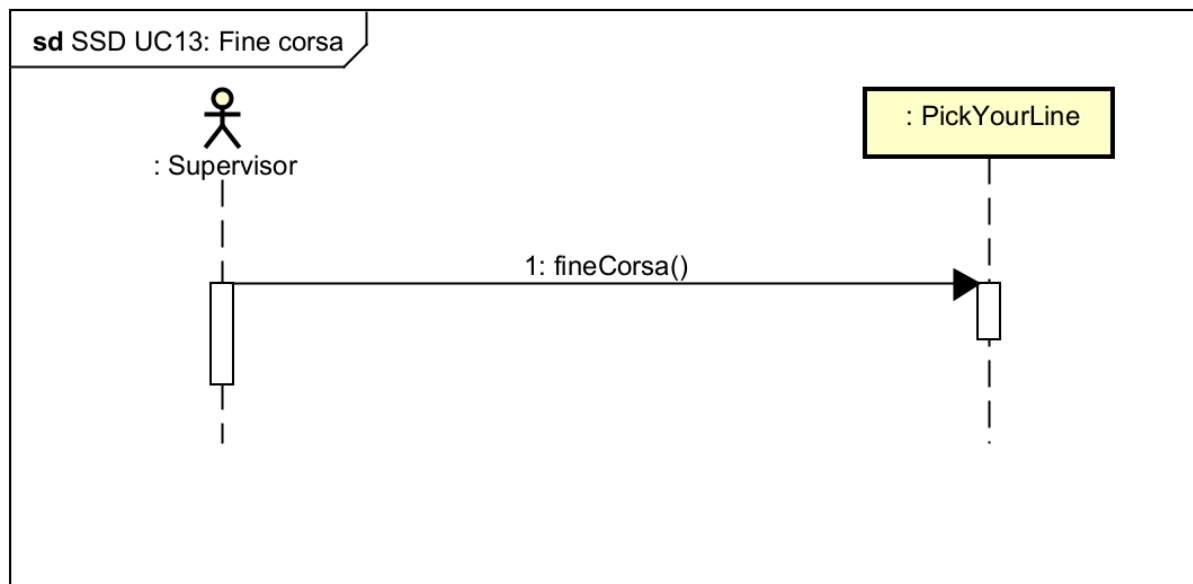


sd SSD UC8: Visualizza Avvisi



sd SSD UC11: Gestisci controllori





Contratti UC7:

Operazione	inserisciAvviso(oggetto, contenuto)
Riferimenti	UC7: Gestisci avvisi
Pre-condizioni	-
Post-condizioni	- viene creata un'istanza av di Avviso - vengono settati gli attributi di av - av viene aggiunta all'elenco degli avvisi

Operazione	modificaAvviso(codice, oggetto, contenuto)
Riferimenti	UC7: Gestisci avvisi

Pre-condizioni	-
Post-condizioni	- viene recuperata un'istanza av di Avviso sulla base del codice - vengono aggiornati gli attributi di av

Operazione	eliminaAvviso(codice)
Riferimenti	UC7: Gestisci avvisi
Pre-condizioni	-
Post-condizioni	- viene recuperata un'istanza av di Avviso sulla base del codice - av viene eliminata dall'elenco degli avvisi

Contratti UC8:

Operazione	visualizzaElencoAvvisi()
Riferimenti	UC8: Visualizza avvisi
Pre-condizioni	Esiste l'elenco degli avvisi
Post-condizioni	viene recuperato l'elenco delle istanze di Avviso

Operazione	visualizzaDettaglioAvviso(codice)
Riferimenti	UC8: Visualizza avvisi
Pre-condizioni	È in corso la visualizzazione degli avvisi
Post-condizioni	viene recuperata l'istanza av di Avviso sulla base del codice

Contratti UC11:

Operazione	visualizzaElencoControllori()
Riferimenti	UC11: Gestisci controllori
Pre-condizioni	Esiste l'elenco dei controllori
Post-condizioni	viene recuperato l'elenco delle istanze di Controllore

Operazione	inserisciControllore(codice)
------------	------------------------------

Riferimenti	UC11: Gestisci controllori
Pre-condizioni	-
Post-condizioni	- viene creata un'istanza co di Controllore - co viene aggiunto all'elenco dei controllori

Operazione	eliminaControllore(codice)
Riferimenti	UC11: Gestisci controllori
Pre-condizioni	-
Post-condizioni	- viene recuperata l'istanza co di C sulla base del codice - co viene eliminata dall'elenco dei controllori

Contratti UC12:

Operazione	visualizzaElencoAutomezziNonInTransito()
Riferimenti	UC12: Inizio corsa
Pre-condizioni	Devono esserci automezzi non in transito
Post-condizioni	Viene recuperato l'elenco delle istanze di Automezzo con stato NonInTransito

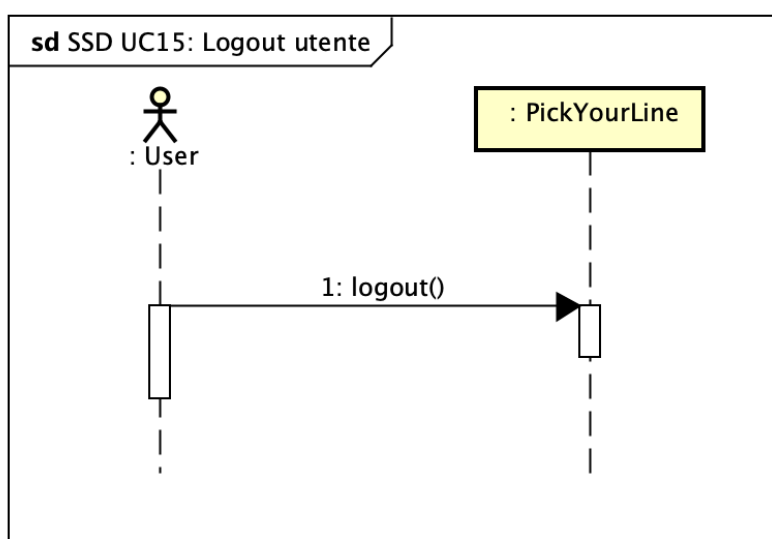
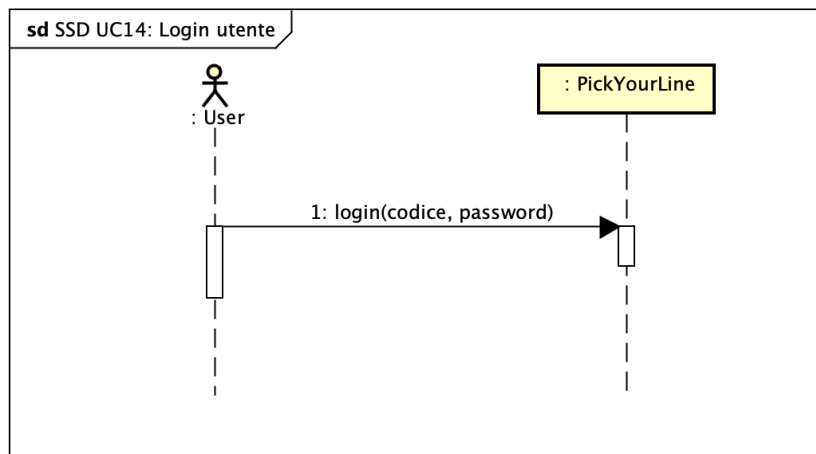
Operazione	supervisionaAutomezzo(codice)
Riferimenti	UC12: Inizio corsa
Pre-condizioni	È in corso l'operazione di inizio corsa
Post-condizioni	- viene recuperata l'istanza co relativa all'utente corrente - viene recuperata l'istanza a di Automezzo sulla base del codice - viene settato l'attributo automezzoSupervisionato di co

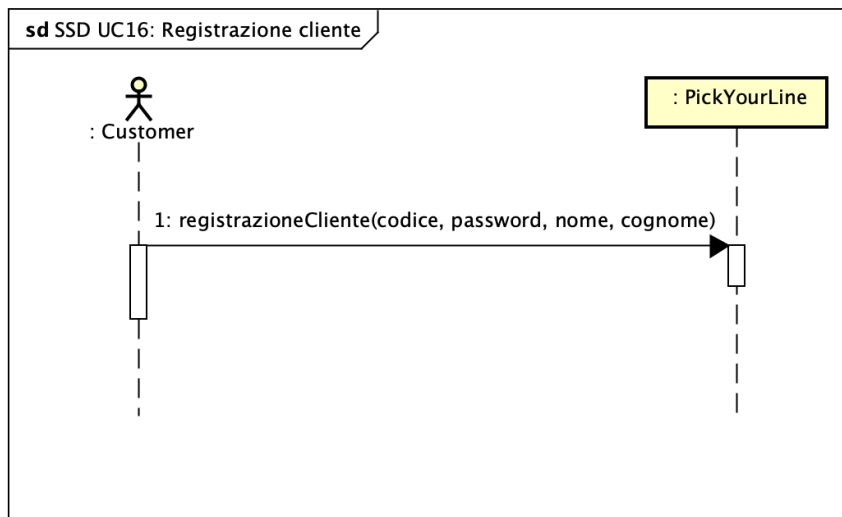
	- vengono aggiornati gli attributi di a
--	---

Contratti UC13:

Operazione	fineCorsa()
Riferimenti	UC13: Fine corsa
Pre-condizioni	Il Controllore deve avere un mezzo supervisionato associato
Post-condizioni	- viene recuperata l'istanza co relativa all'utente corrente - vengono aggiornati gli attributi di a - l'associazione tra co e l'automezzo associato viene sciolta

Iterazione 5:





Contratti UC14:

Operazione	login(email, password)
Riferimenti	UC14: Login utente
Pre-condizioni	L'Utente deve essere registrato
Post-condizioni	- viene recuperata un'istanza u di Utente sulla base delle credenziali - u viene associato a PickYourLine tramite l'associazione utenteCorrente

Contratti UC15:

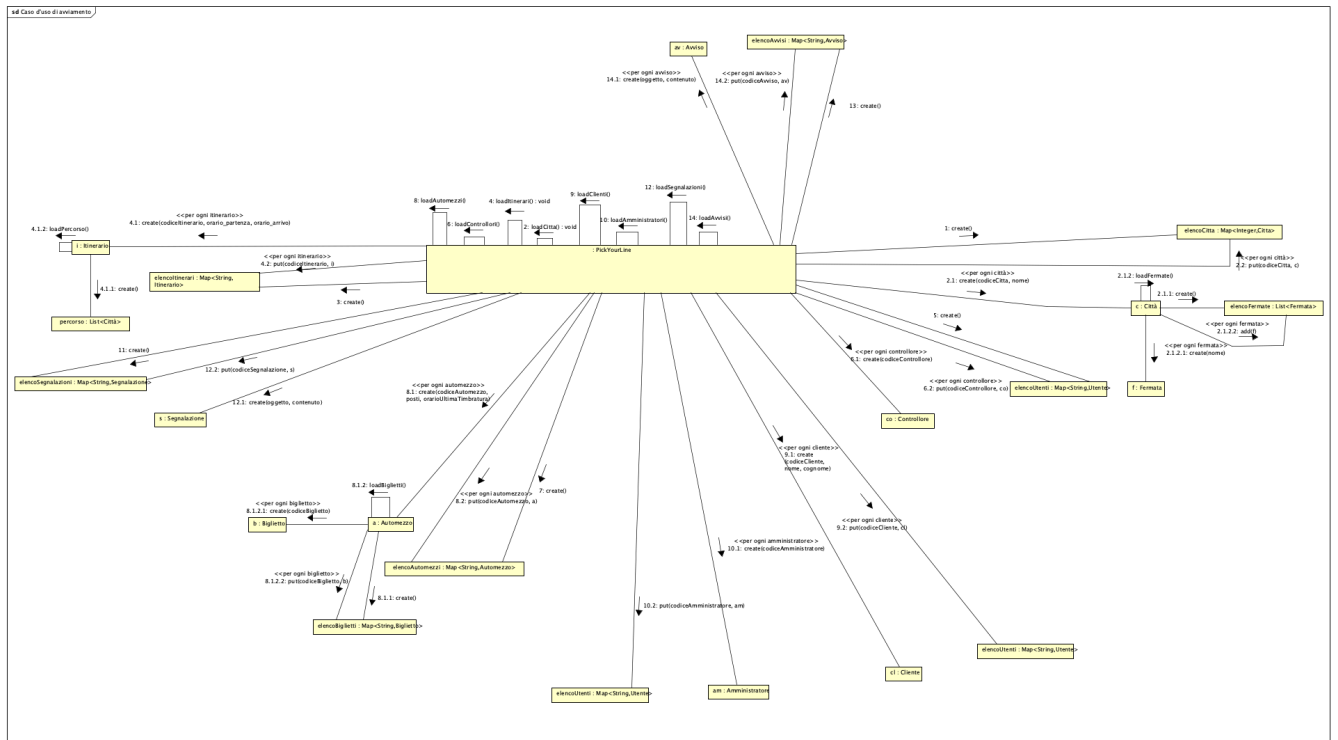
Operazione	logout()
Riferimenti	UC15: Logout utente
Pre-condizioni	L'Utente deve essere autenticato
Post-condizioni	- l'associazione tra u e PickYourLine viene sciolta

Contratti UC16:

Operazione	registrazioneCliente(codice, password, nome, cognome)
Riferimenti	UC14: Registrazione cliente
Pre-condizioni	-
Post-condizioni	- viene creata un'istanza c di Cliente

- | | |
|--|--|
| | <ul style="list-style-type: none"> - vengono settati gli attributi di c - c viene inserito nell'elenco dei Clienti - c viene associato a PickYourLine tramite l'associazione utenteCorrente |
|--|--|

Caso d'uso d'avviamento



L'elaborato principale di questa fase è il Modello di Progetto, ovvero l'insieme dei diagrammi che descrivono la progettazione logica sia da un punto di vista dinamico (Diagrammi di Interazione) che da un punto di vista statico (Diagramma delle Classi).

[illegible]

Per quanto riguarda la gestione degli stati degli automezzi si è scelto di utilizzare il Pattern GOF **State** in modo da semplificarne la variazione e le transizioni.

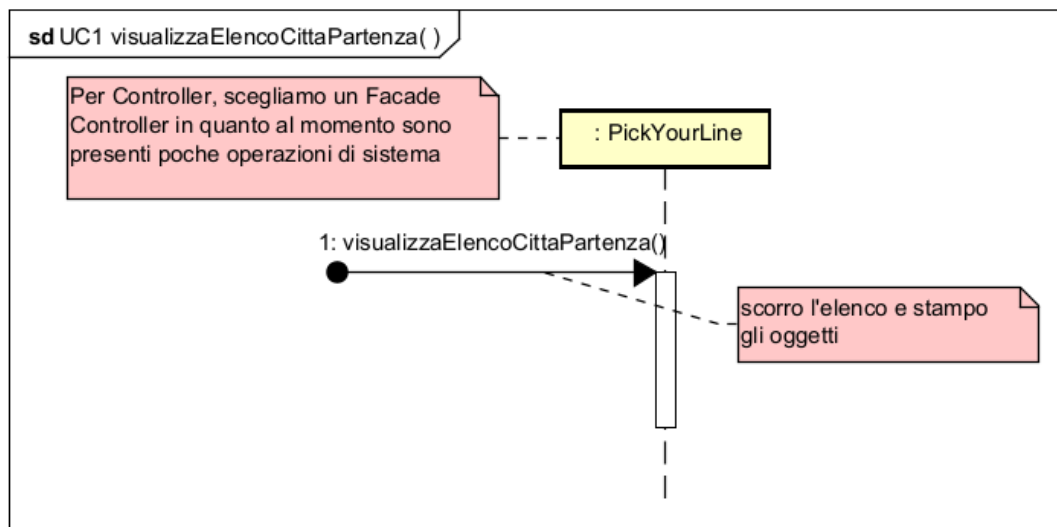
La scelta di questo pattern è stata fatta principalmente per la gestione delle operazioni sugli automezzi in quanto non sarebbe logico poter modificare/eliminare un automezzo in transito, in manutenzione o modificare un automezzo dismesso. Un altro motivo valido che giustifica l'utilizzo di questo pattern è la possibilità di associare delle operazioni alle transizioni di stato, come l'invio automatico di avvisi e operazioni di setup quali il reset dei posti disponibili e l'aggiornamento della posizione.

Diagrammi di interazione

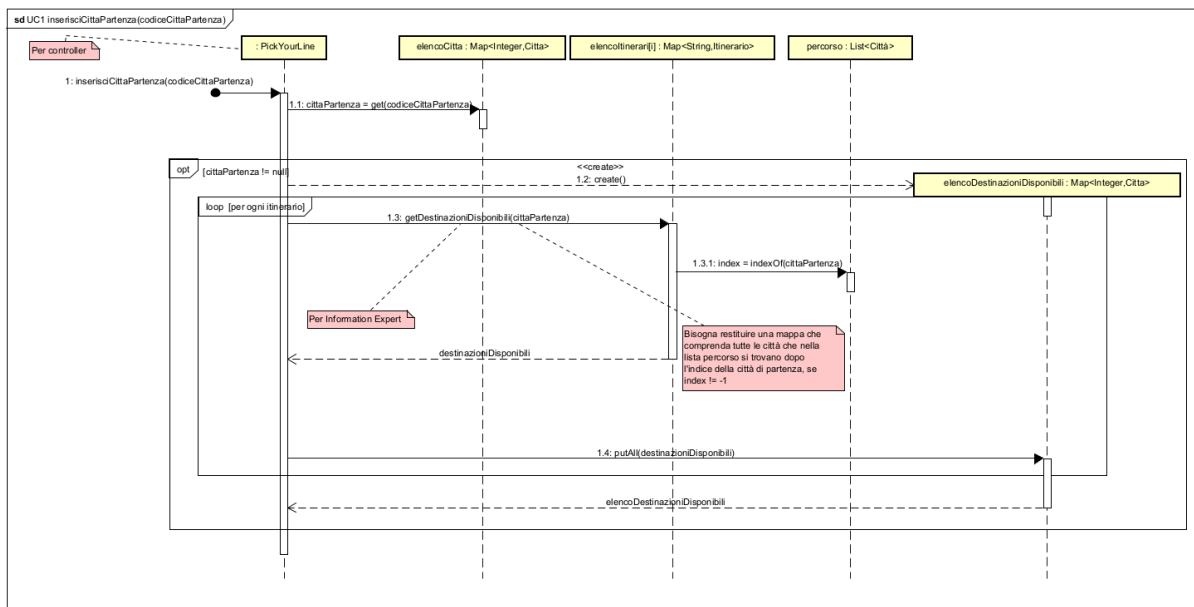
Iterazione 1

UC1

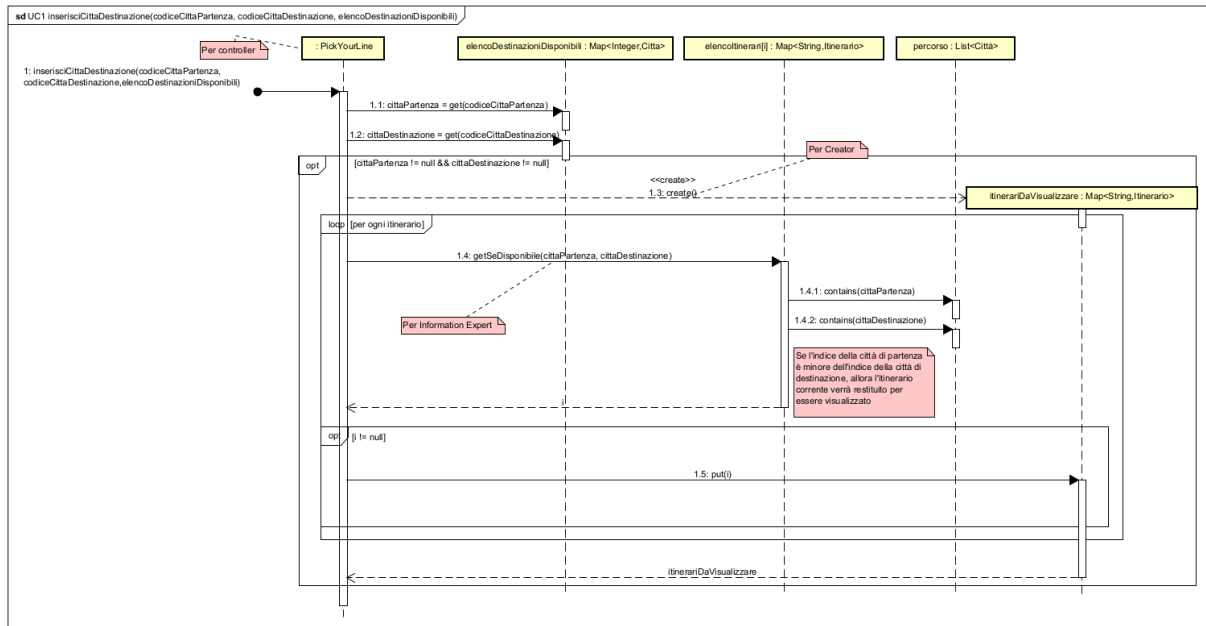
- visualizzaElencoCittaPartenza



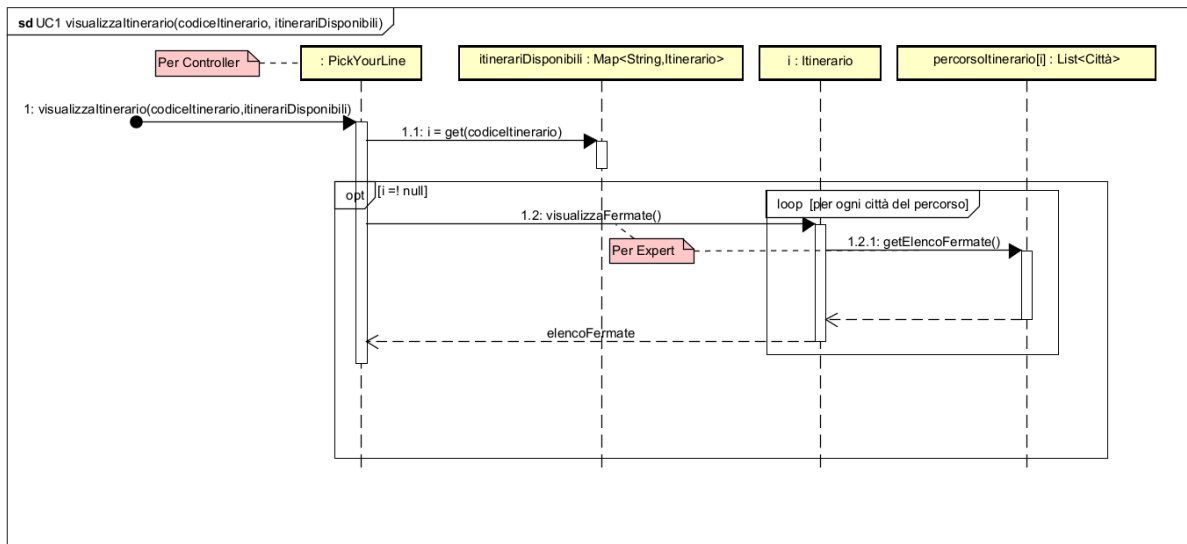
- inserisciCittaPartenza



- inserisciCittaDestinazione



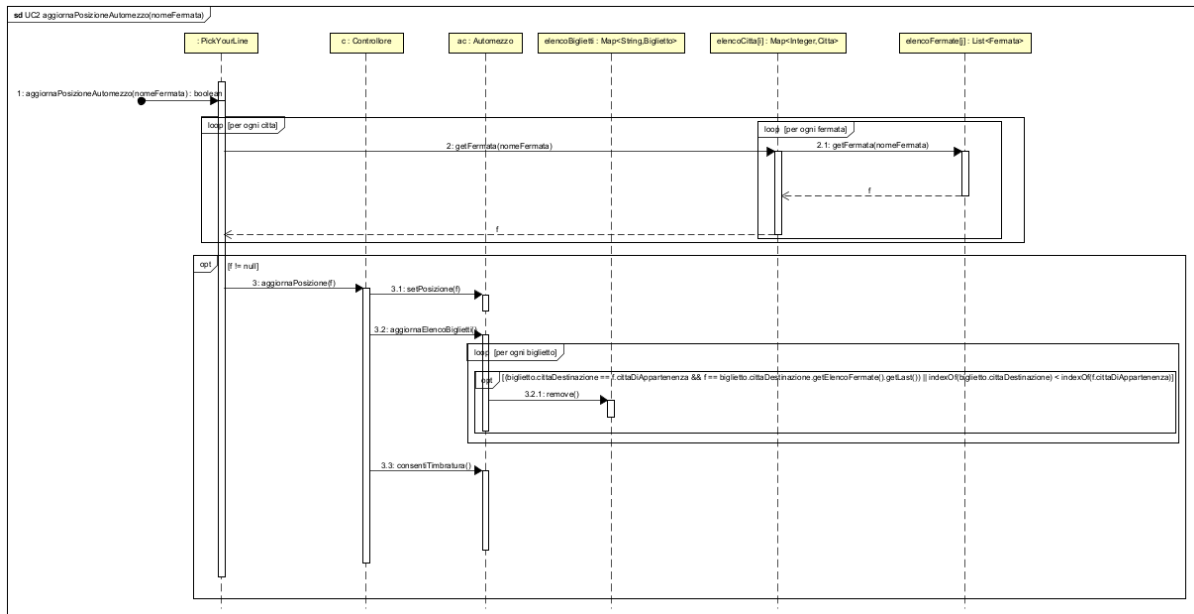
- visualizzaItinerario



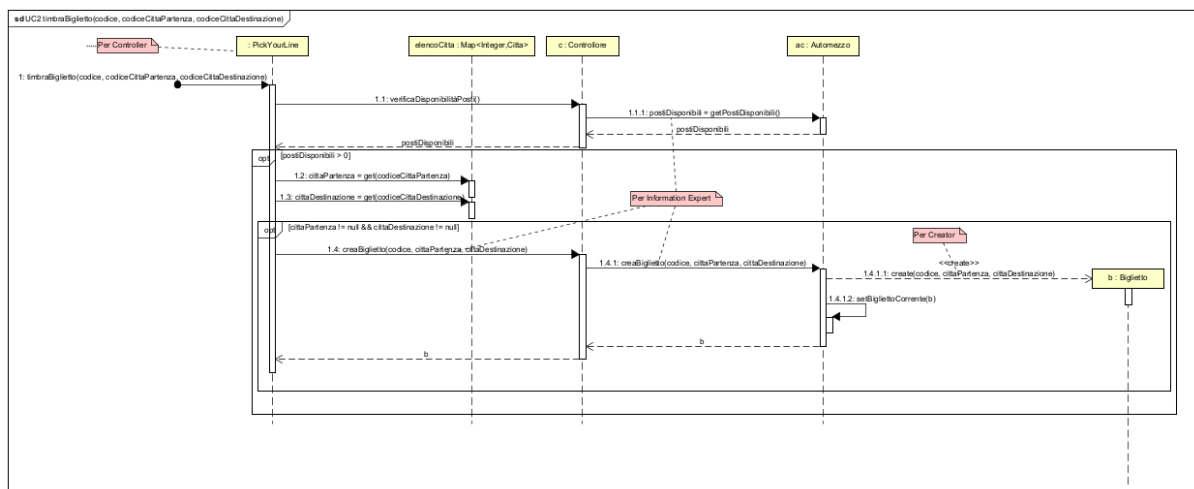
Iterazione 2

UC2

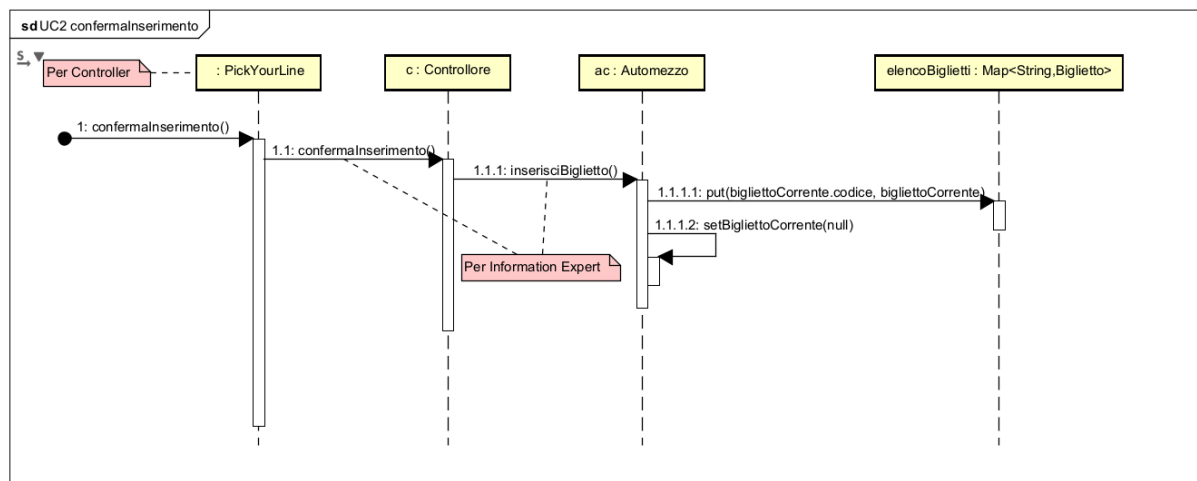
- aggiornaPosizioneAutomezzo



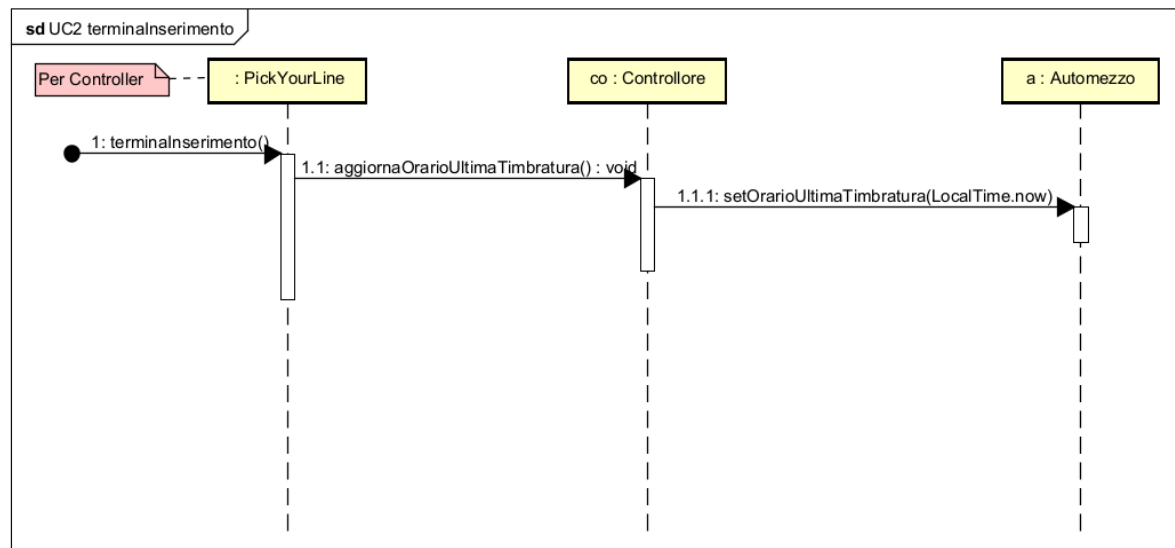
- timbraBiglietto



- confermaInserimento

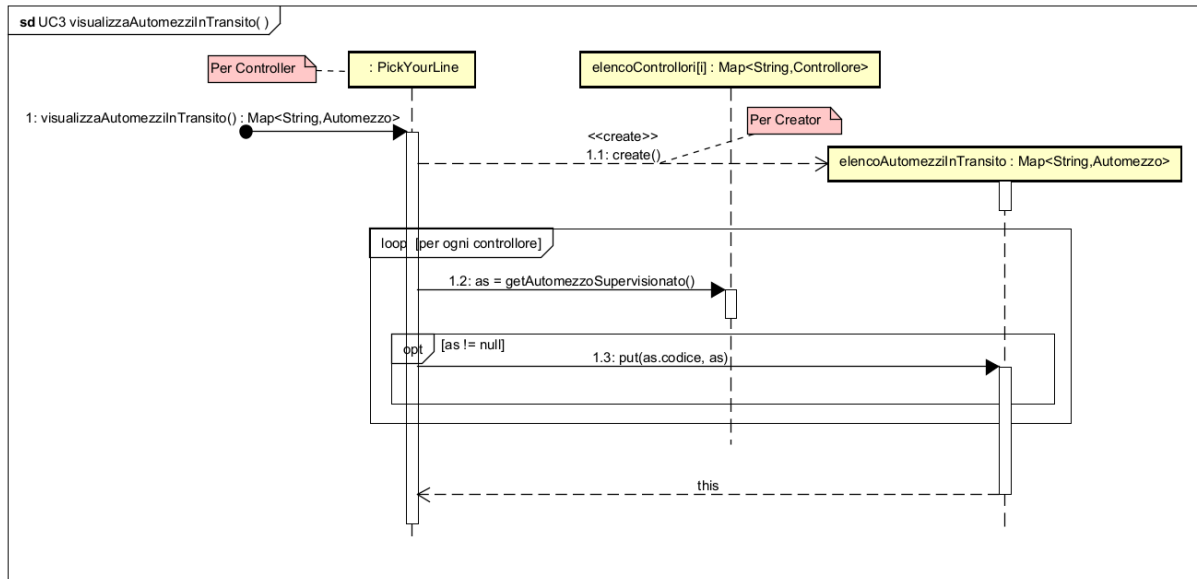


- terminalInserimento

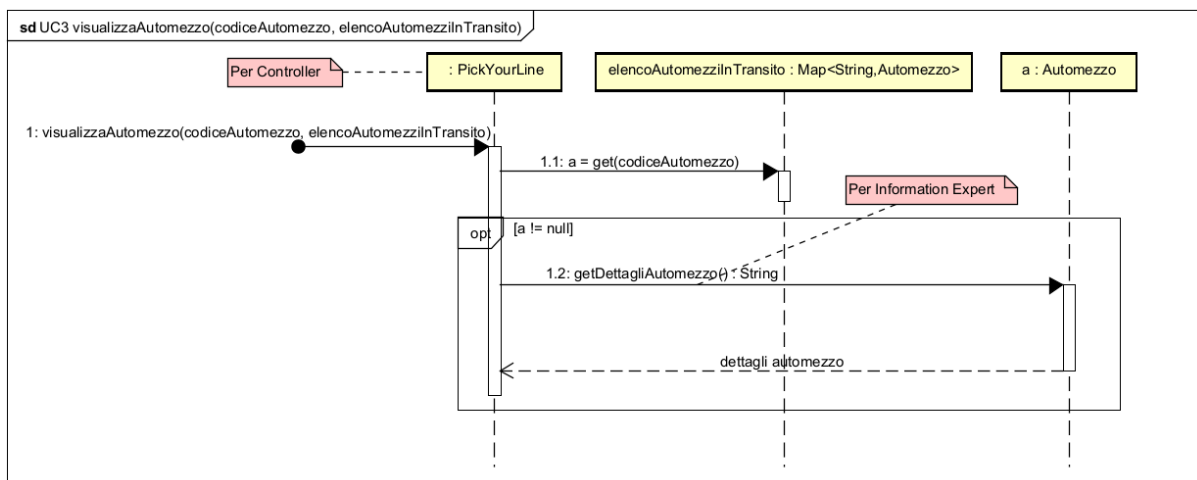


UC3

- visualizzaAutomezzilnTransito



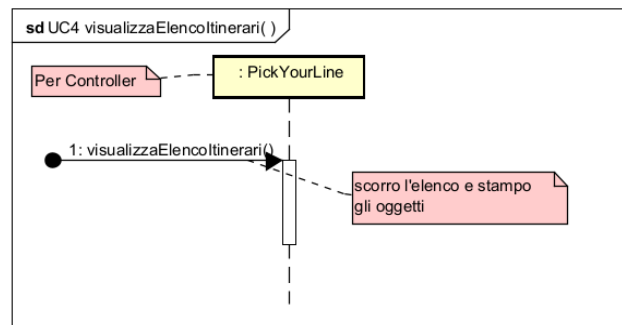
- visualizzaAutomezzo



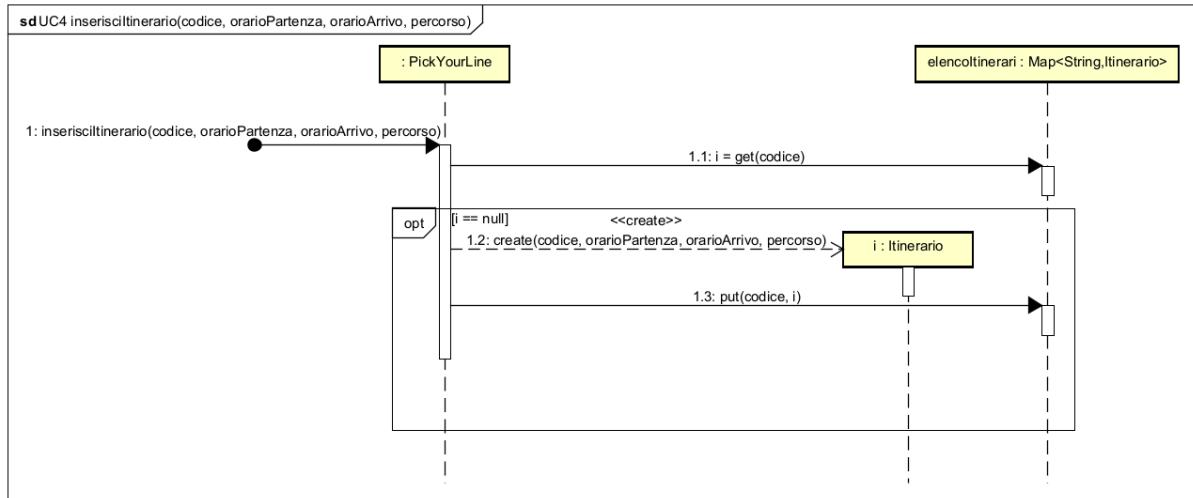
Iterazione 3

UC4

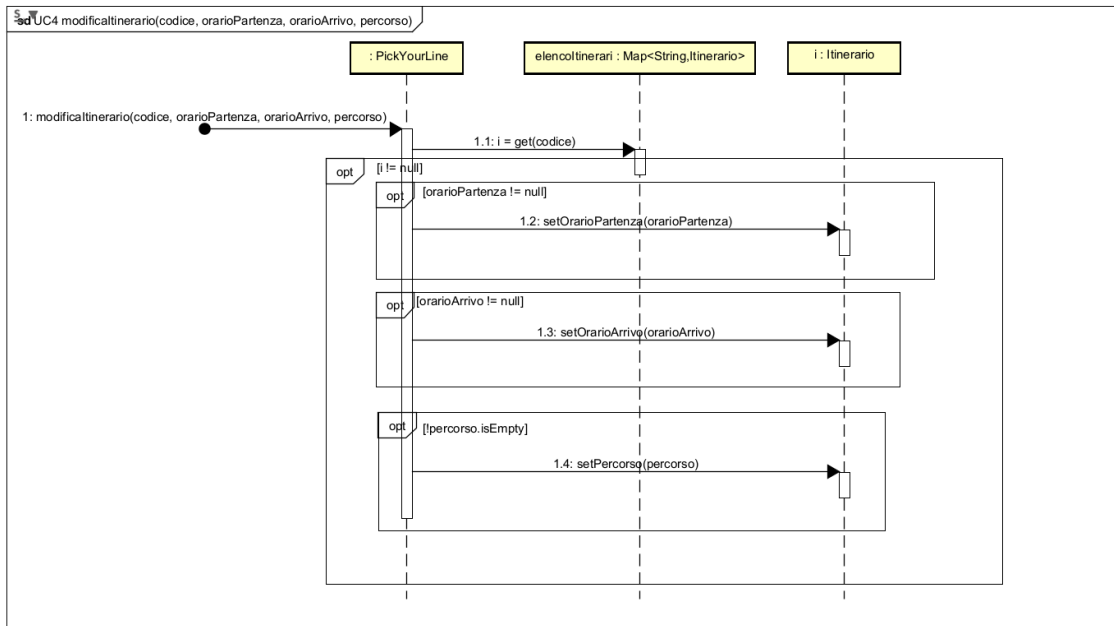
- visualizzaElencoItinerari



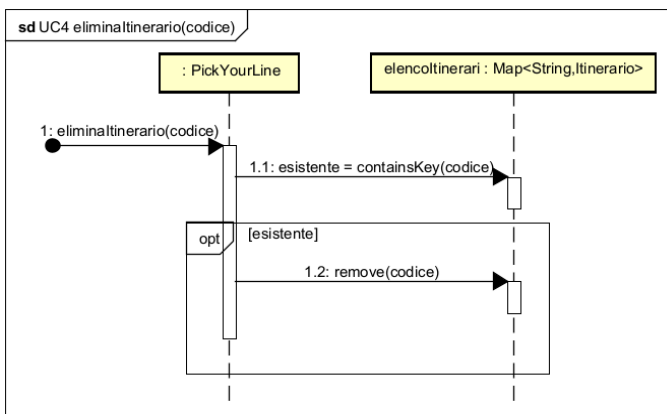
- inserisciltinerario



- modificaltinerario

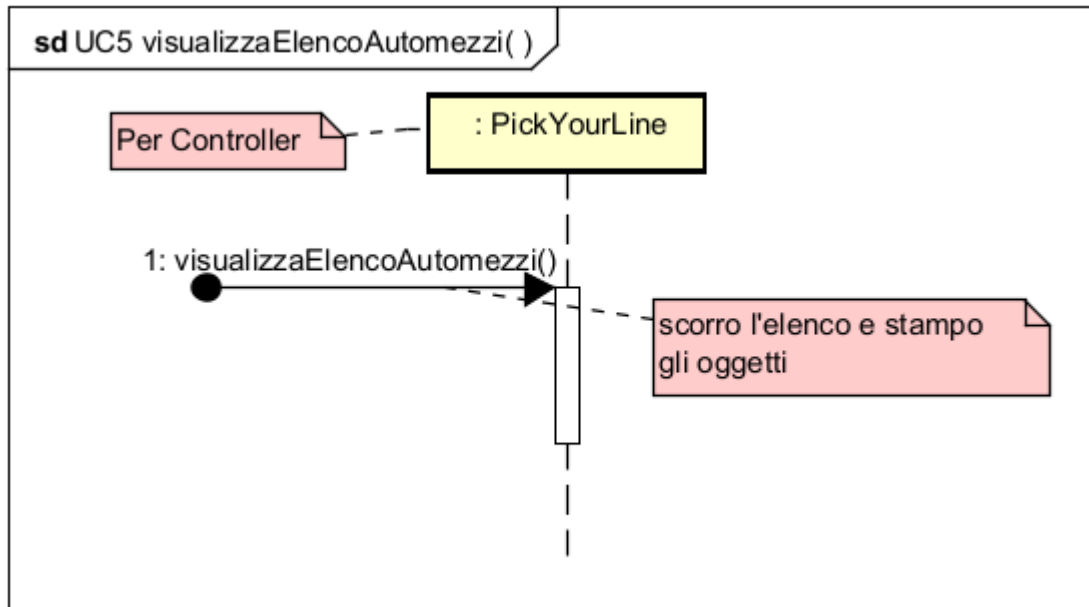


- eliminaltinerario

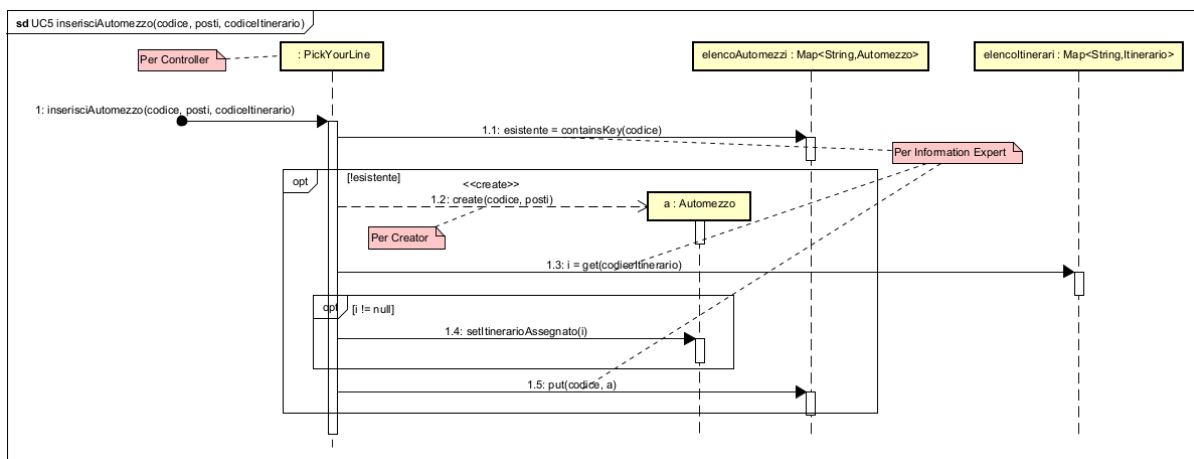


UC5

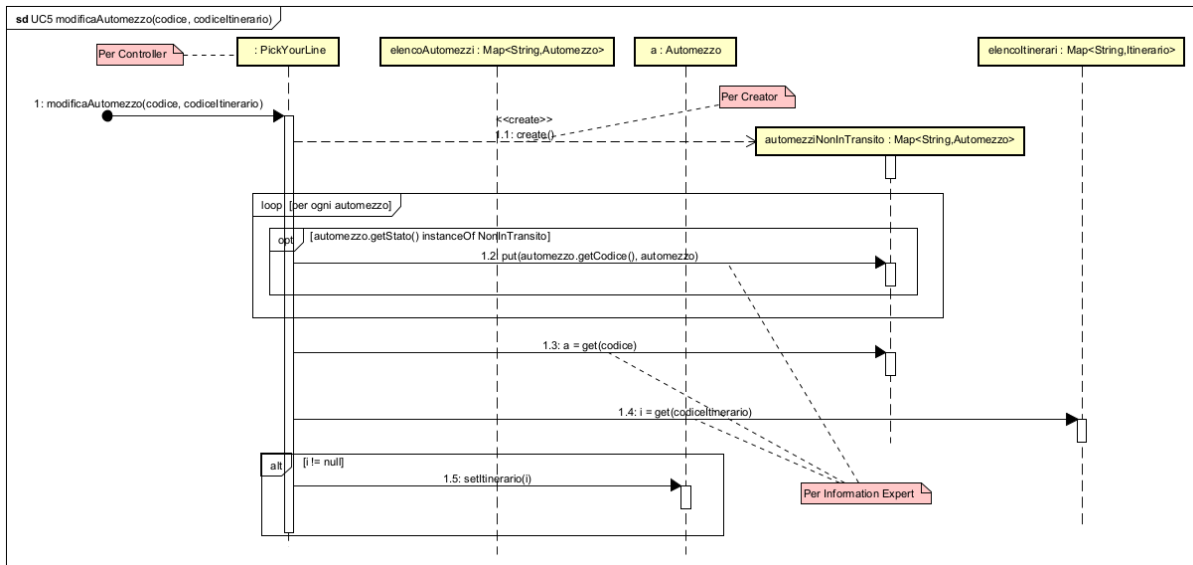
- visualizzaElencoAutomezzi



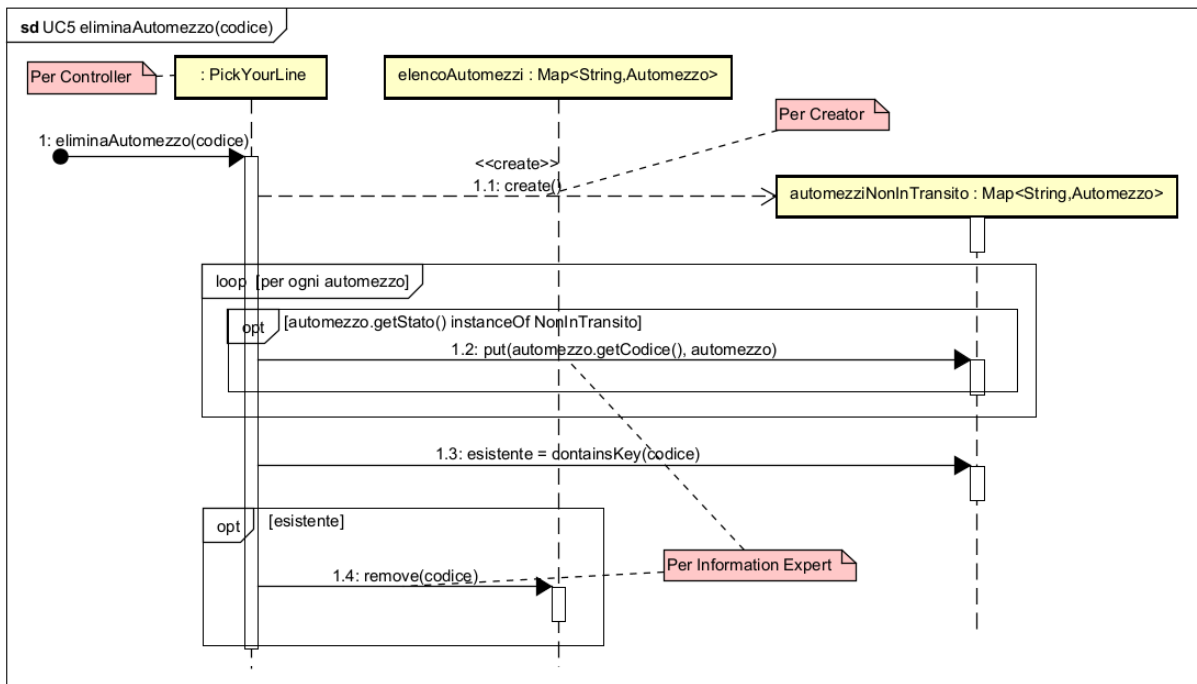
- inserisciAutomezzo



- modificaAutomezzo

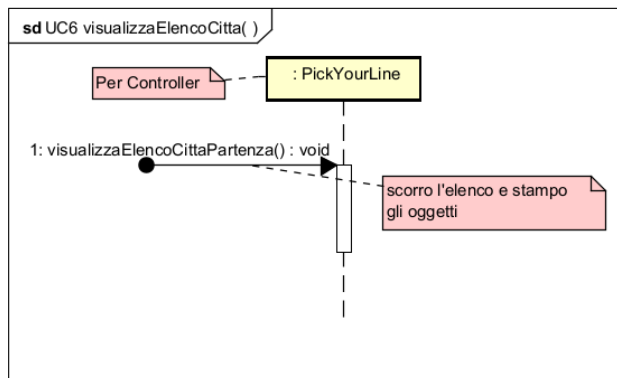


- eliminaAutomezzo

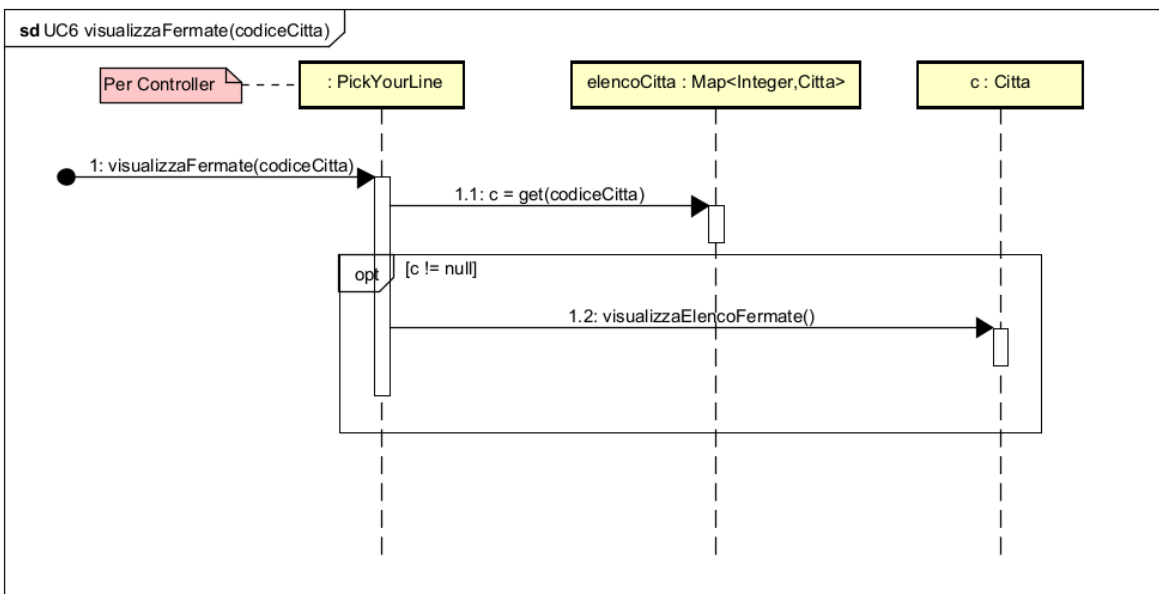


UC6

- visualizzaElencoCitta

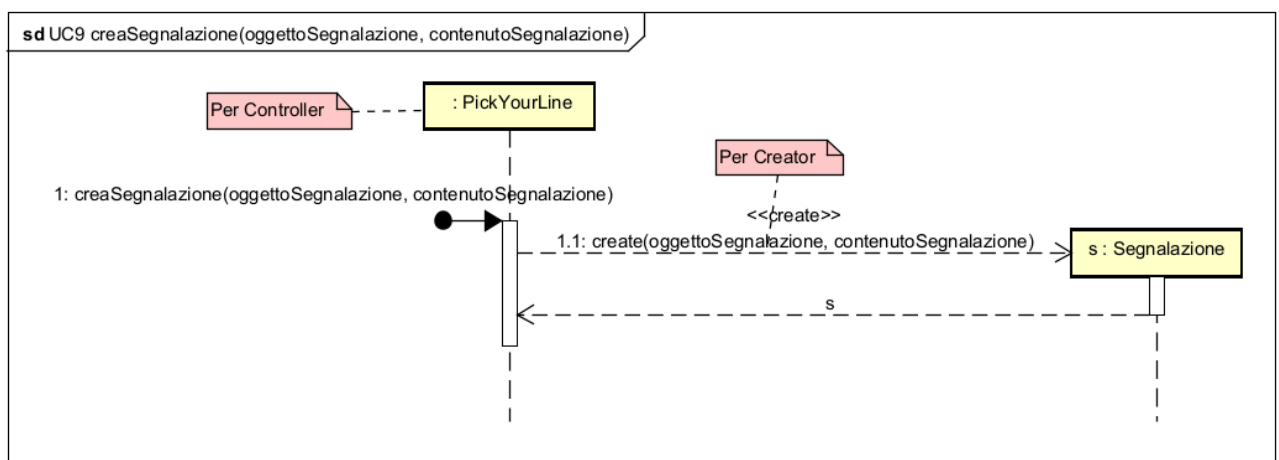


- visualizzaFermate

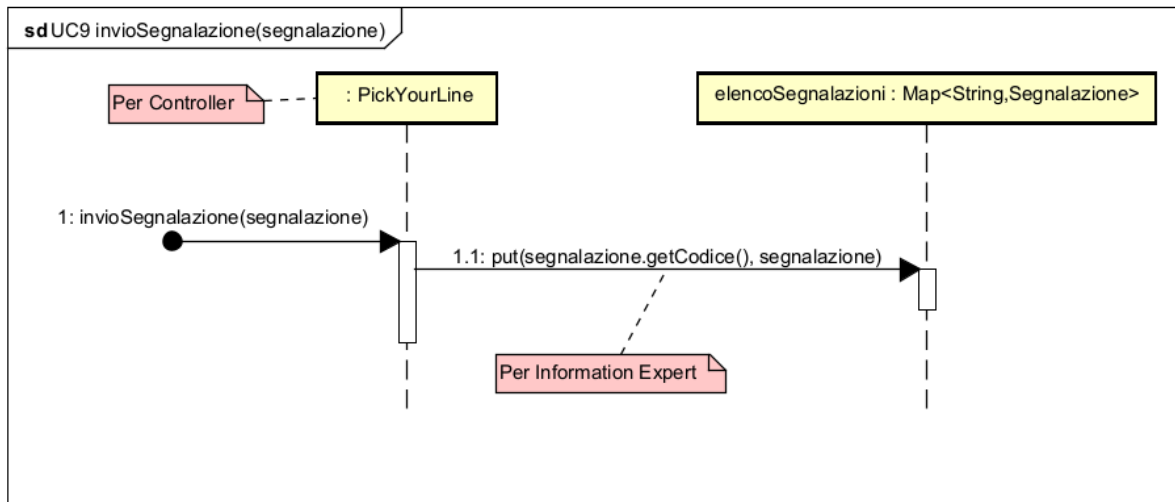


UC9

- creaSegnalazione

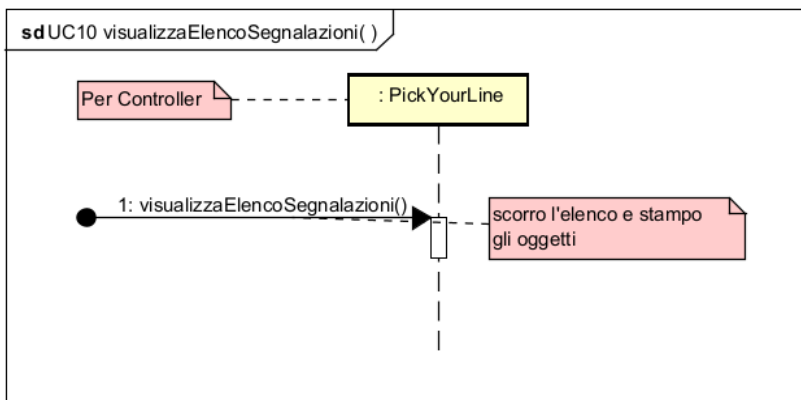


- invioSegnalazione

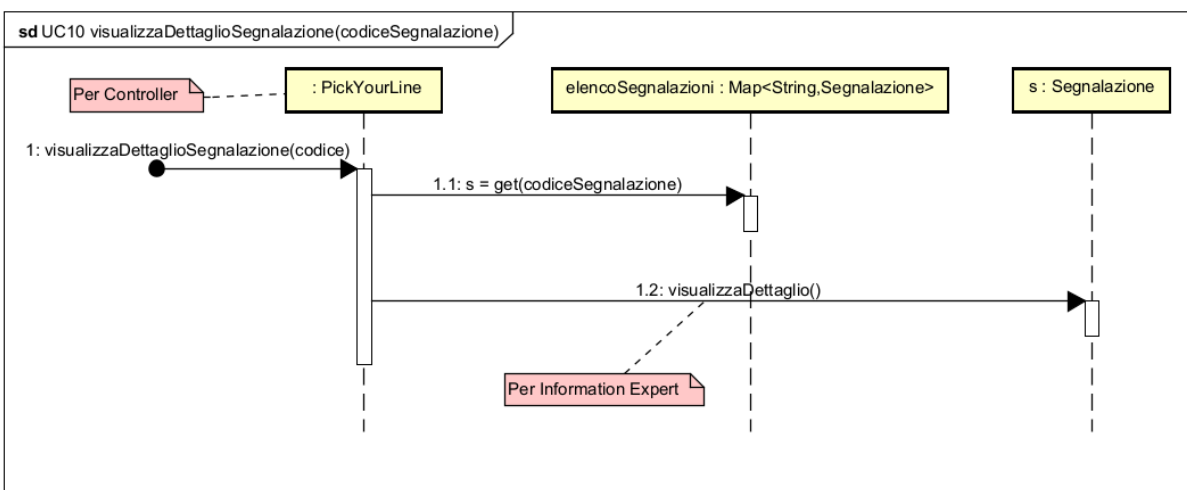


UC10

- visualizzaElencoSegnalazioni

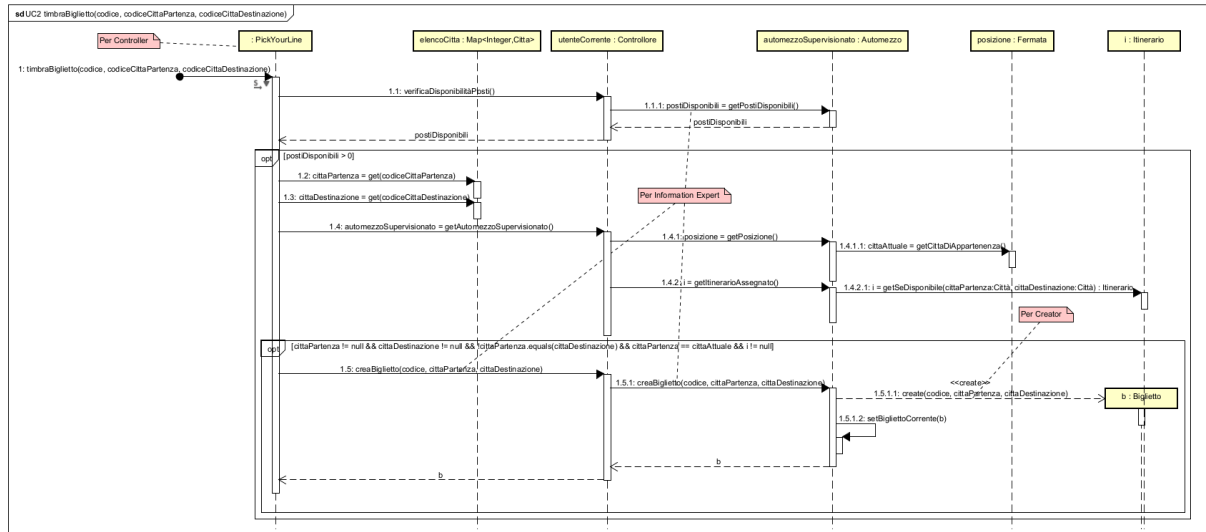
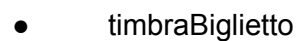


- visualizzaDettaglioSegnalazione



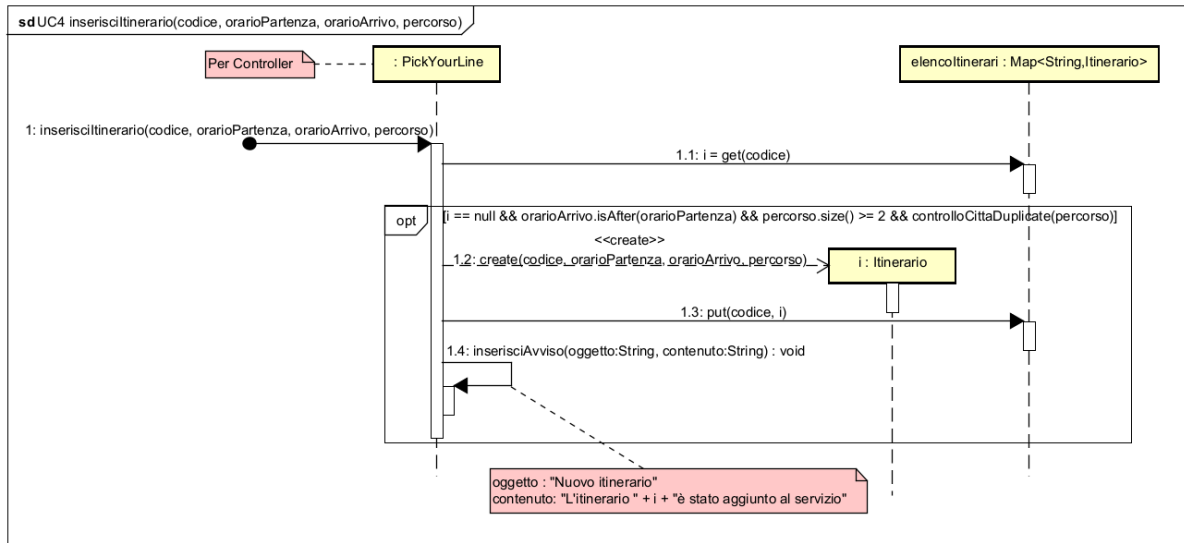
Durante questa iterazione sono stati raffinati i casi d'uso UC2, UC4, UC5 aggiungendo controlli e la generazione automatica di avvisi.

- aggiornaPosizioneAutomezzo

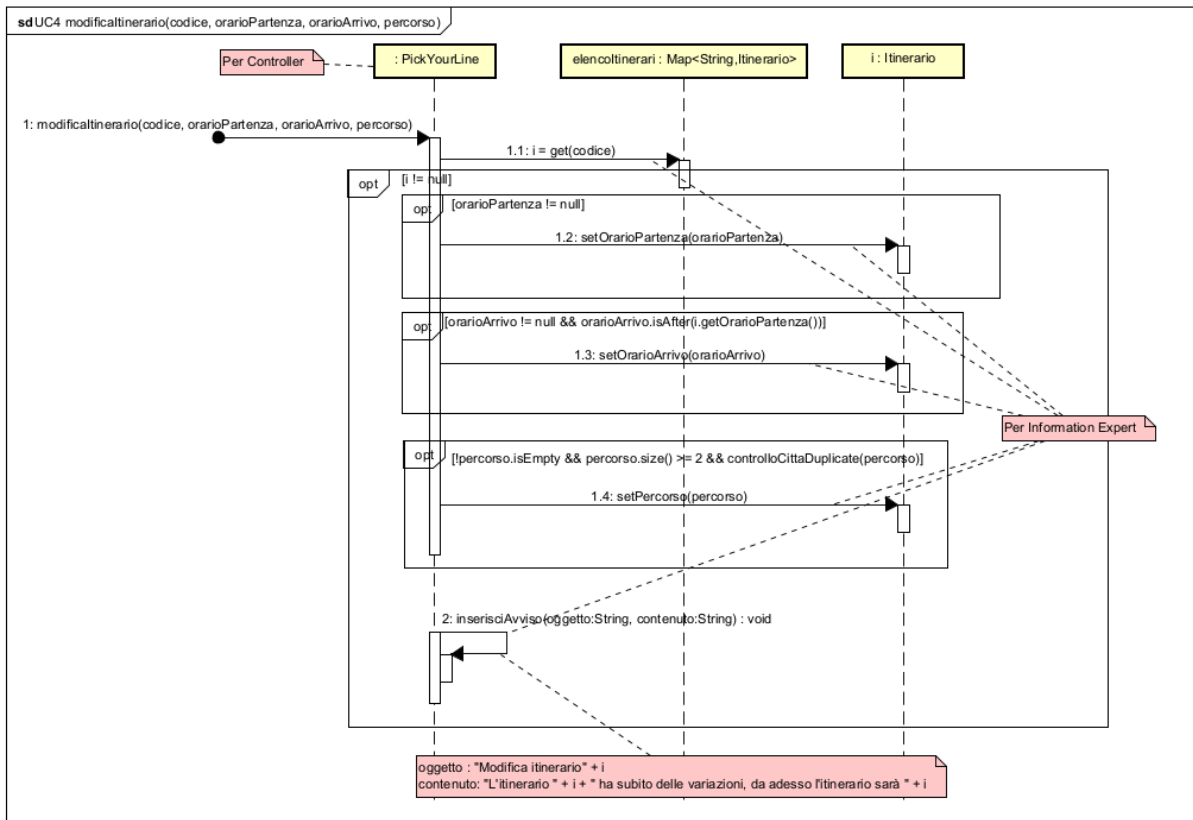


UC4

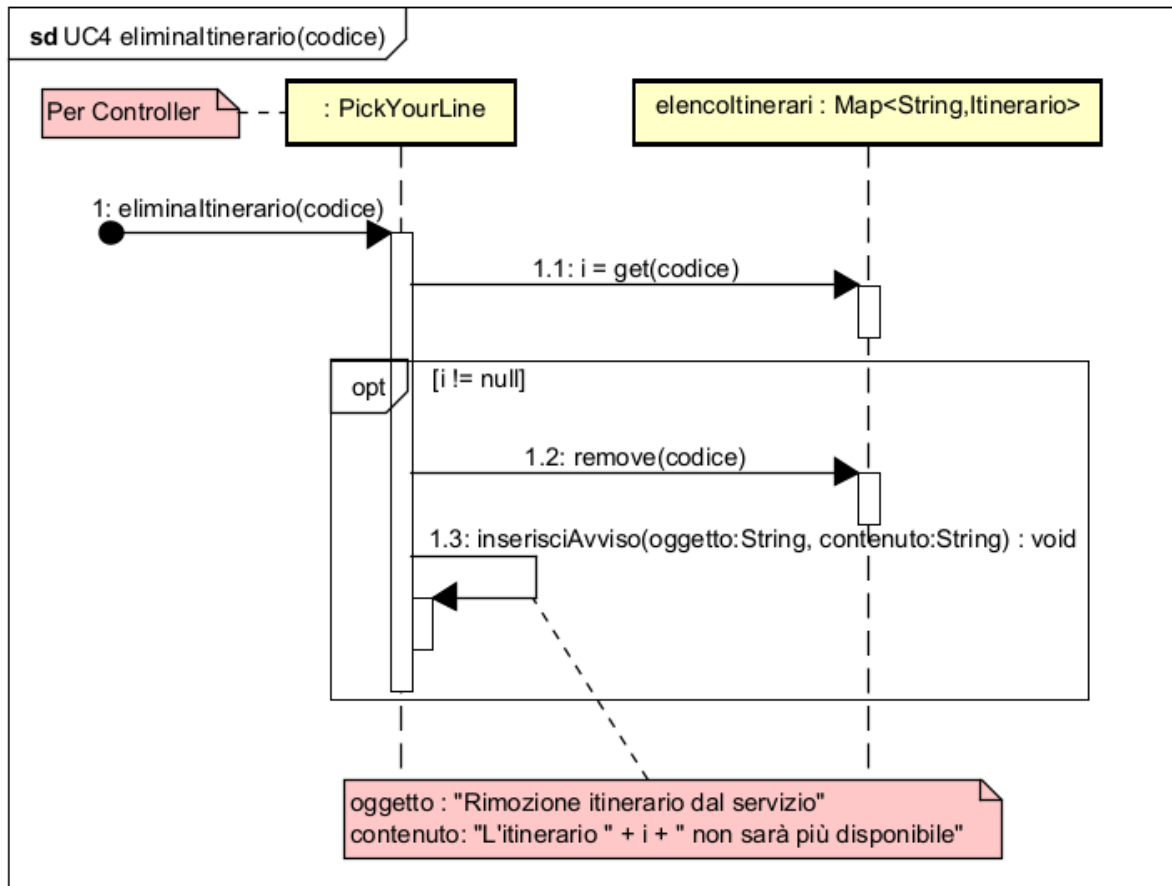
• inserisciltinerario



• modificaltinerario

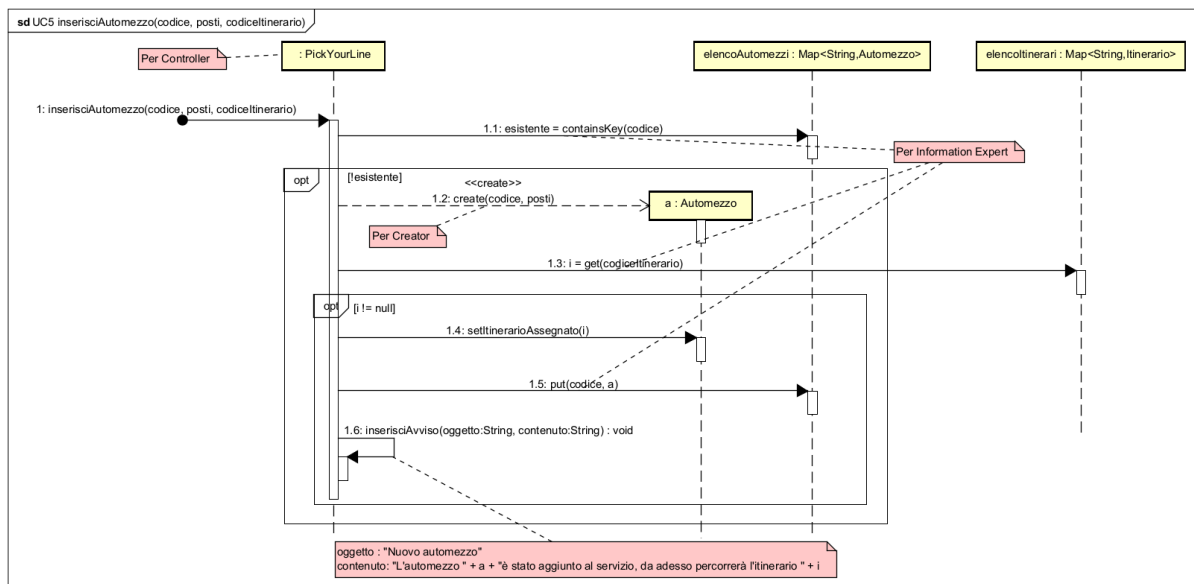


- eliminaitinerario

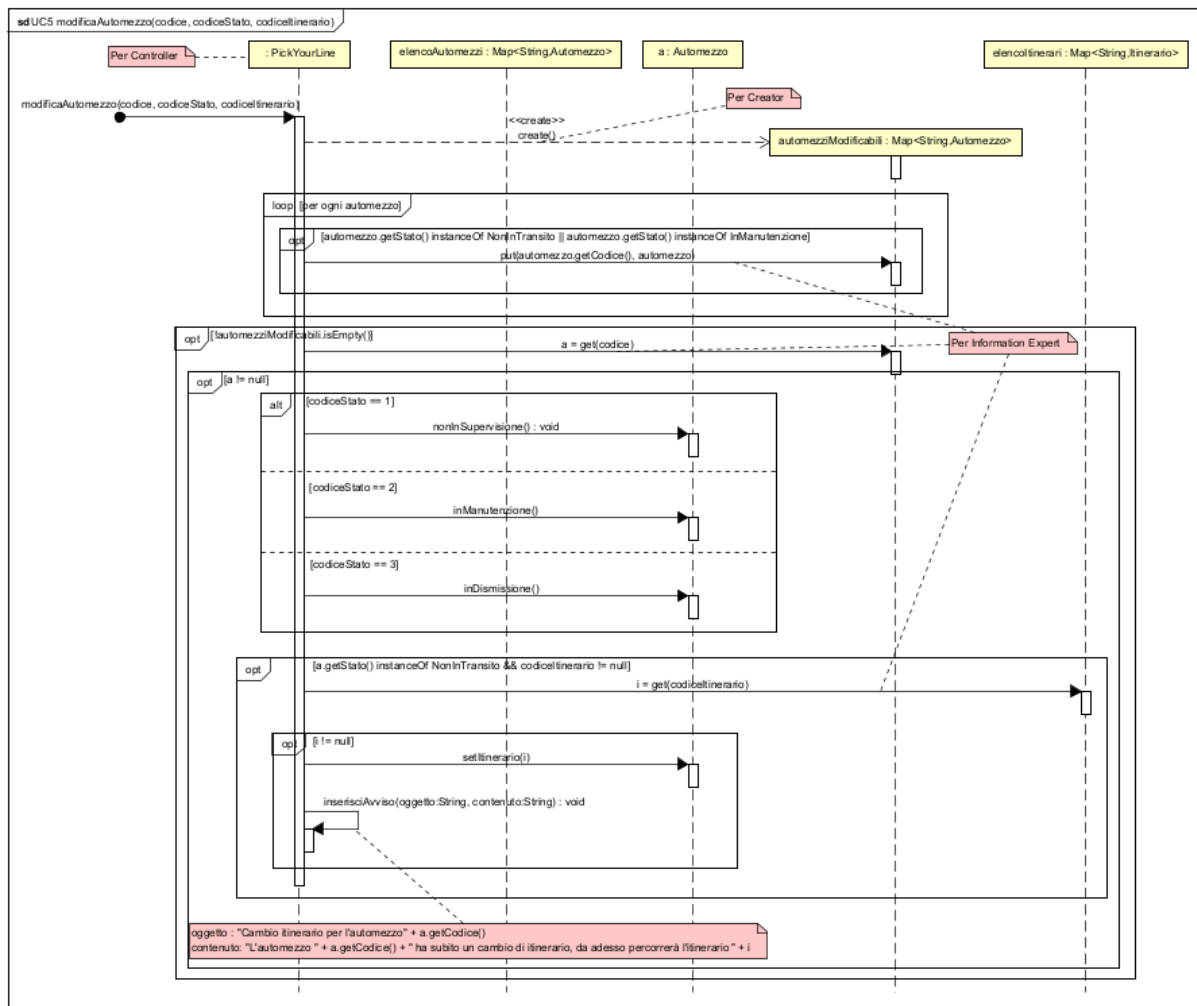


UC5

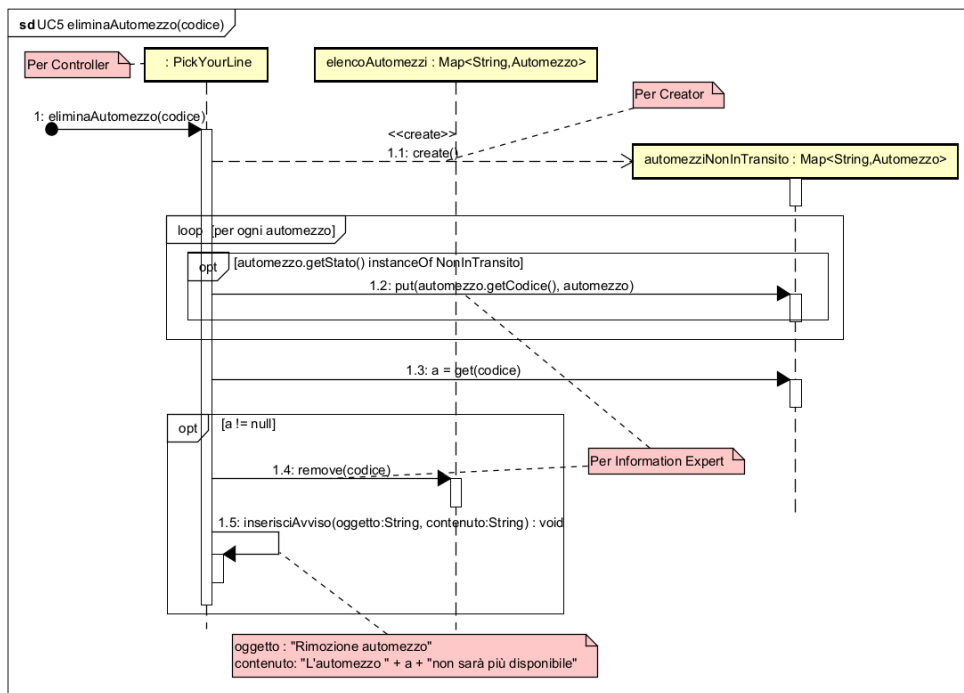
- inserisciAutomezzo



- modificaAutomezzo

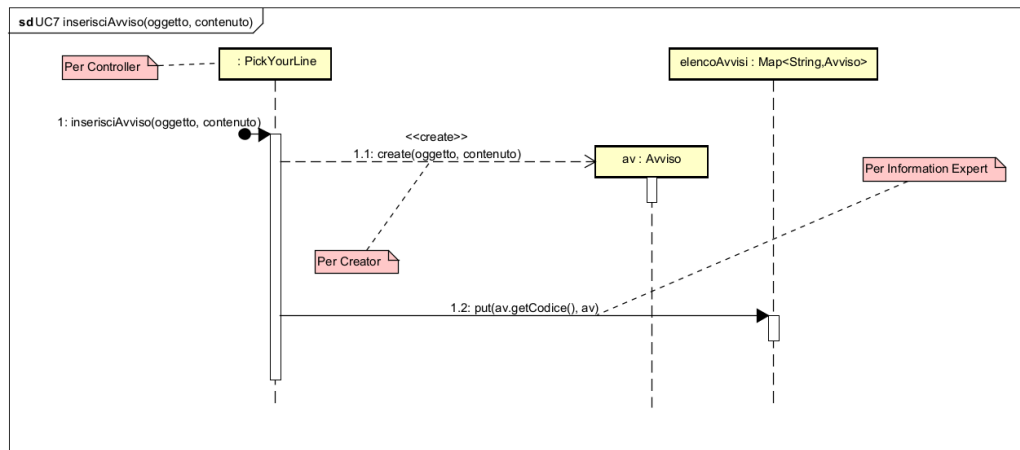


- eliminaAutomezzo

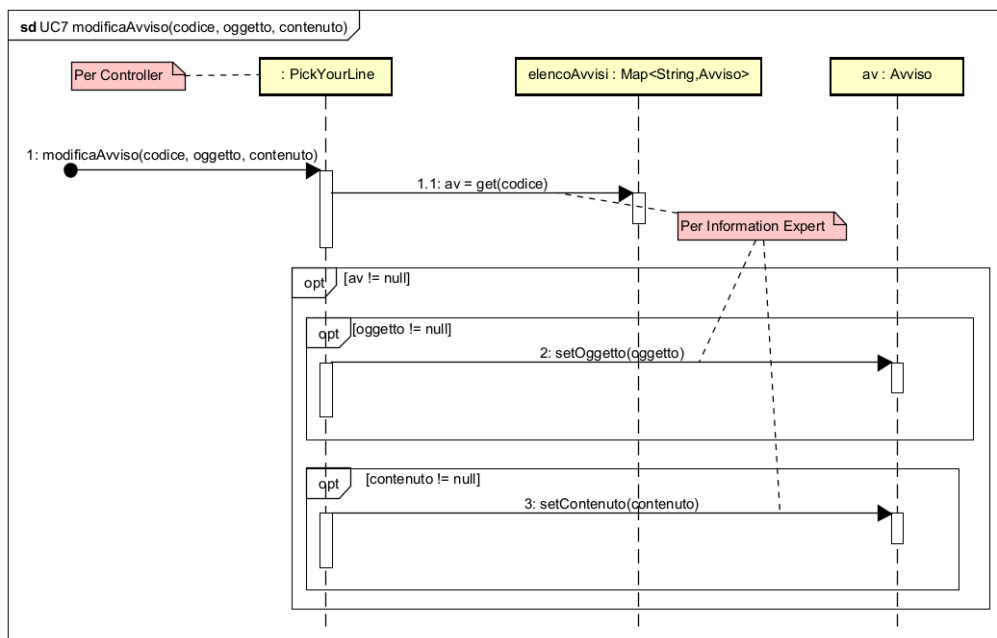


UC7

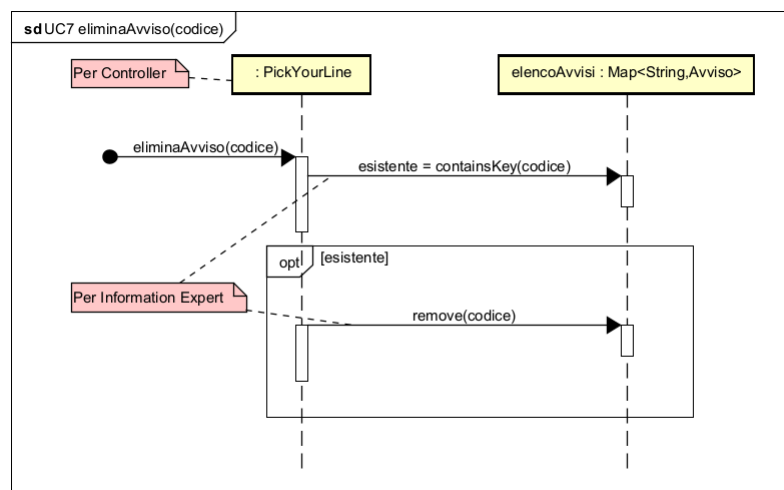
- inserisciAvviso



- modificaAvviso

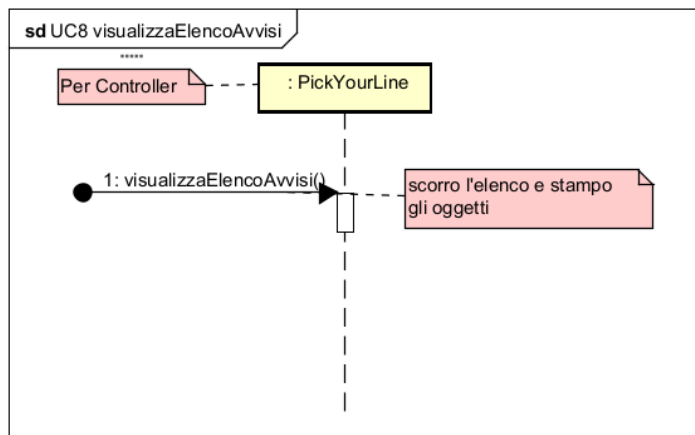


- eliminaAvviso

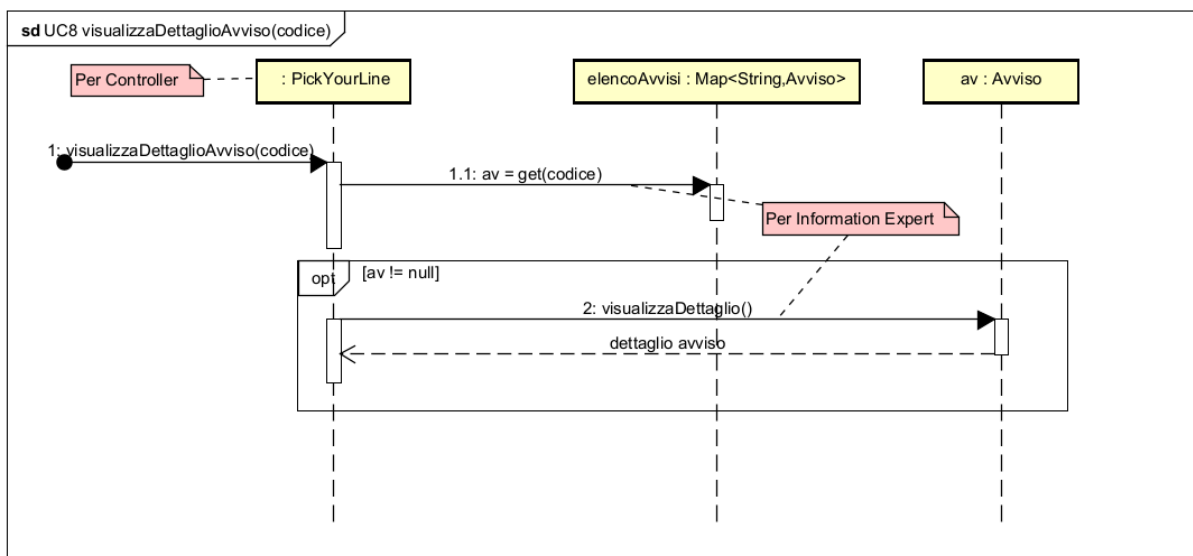


UC8

- visualizzaElencoAvvisi

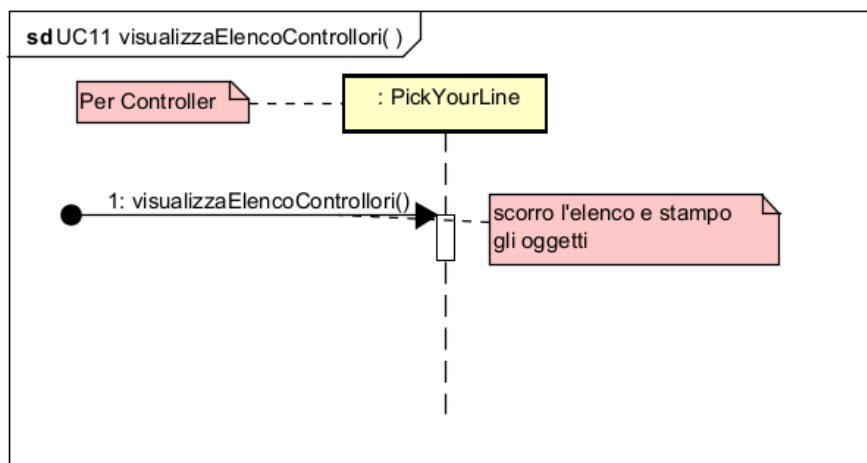


- visualizzaDettaglioAvviso

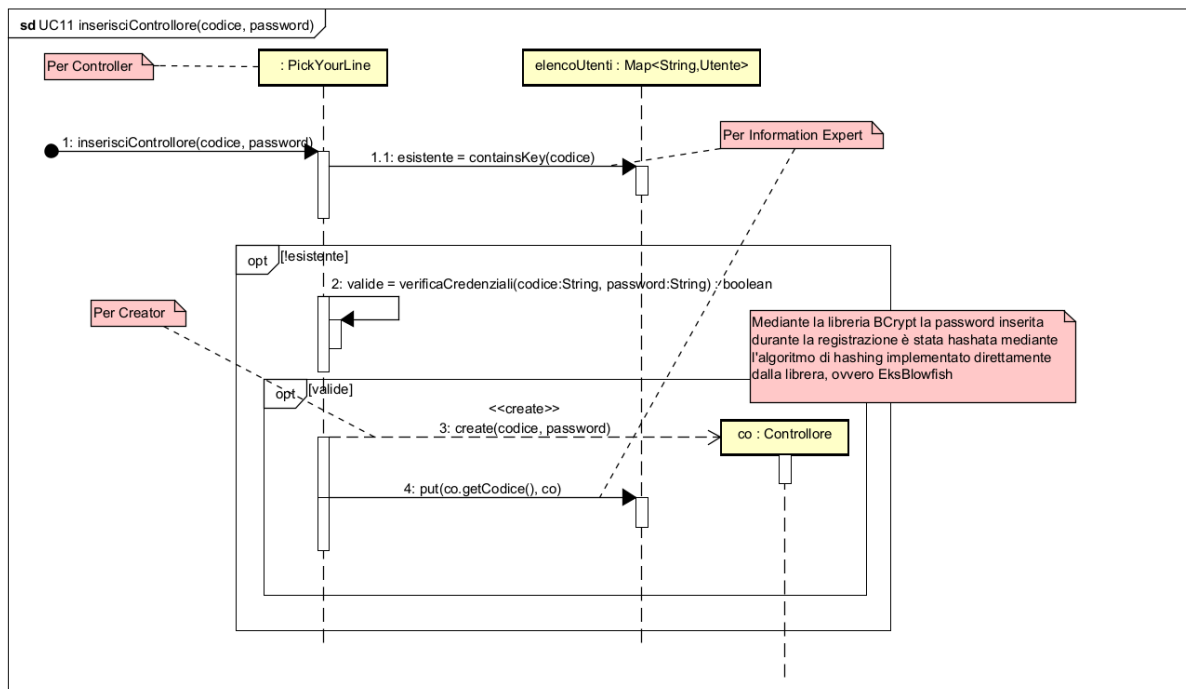


UC11

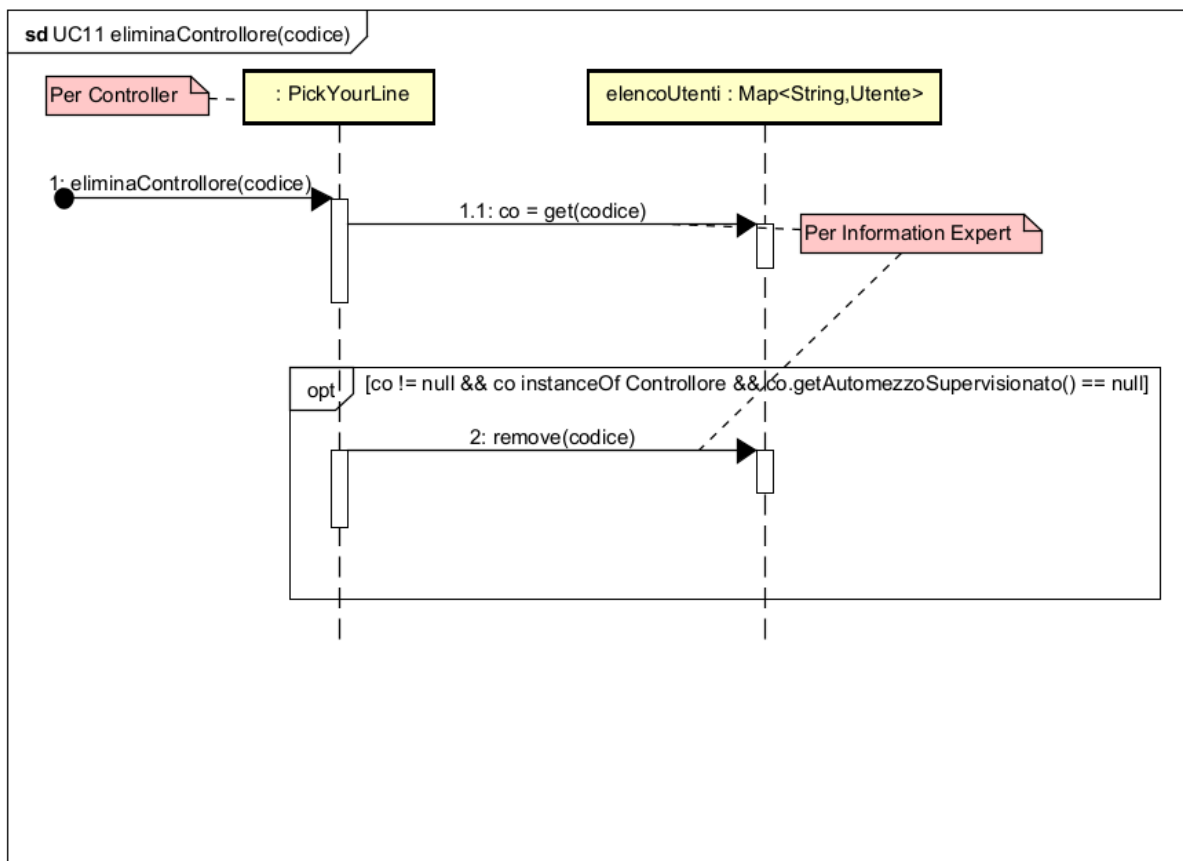
- visualizzaElencoControllori



- inserisciControllore

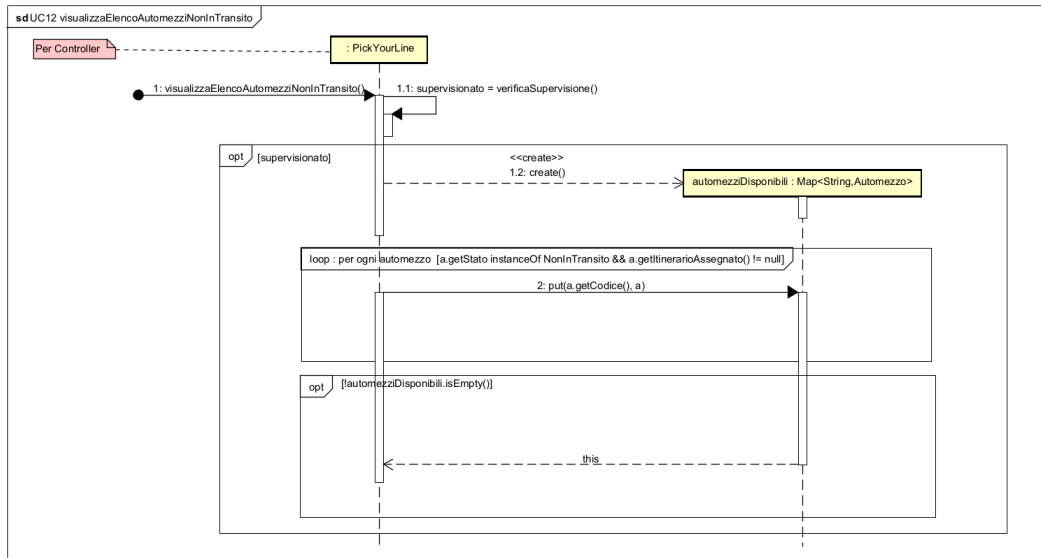


- eliminaControllore

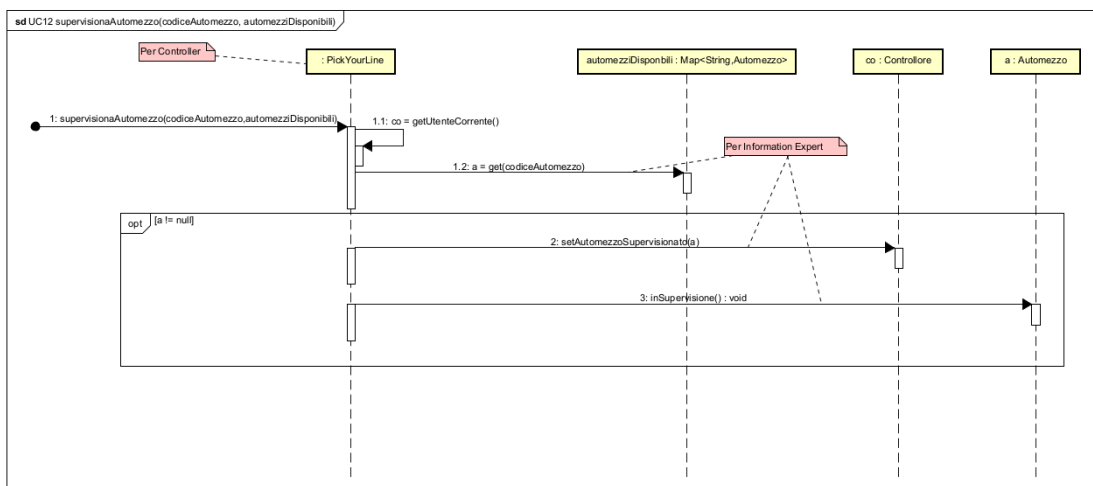


UC12

• visualizzaElencoAutomezziNonInTransito

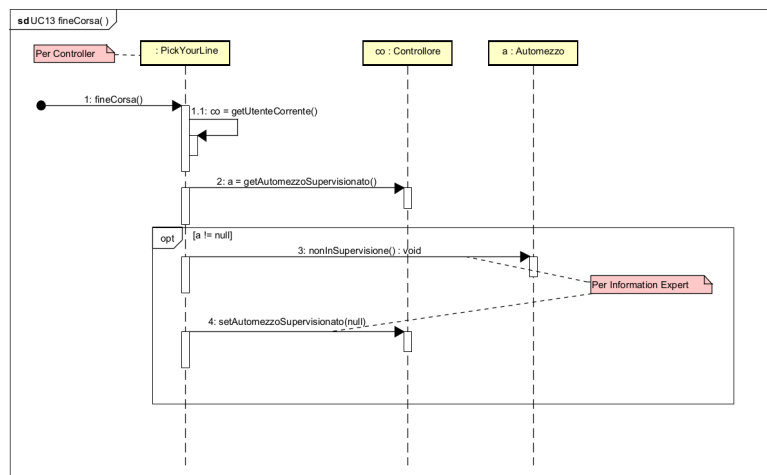


• supervisionaAutomezzo



UC13

• fineCorsa



Iterazione 5

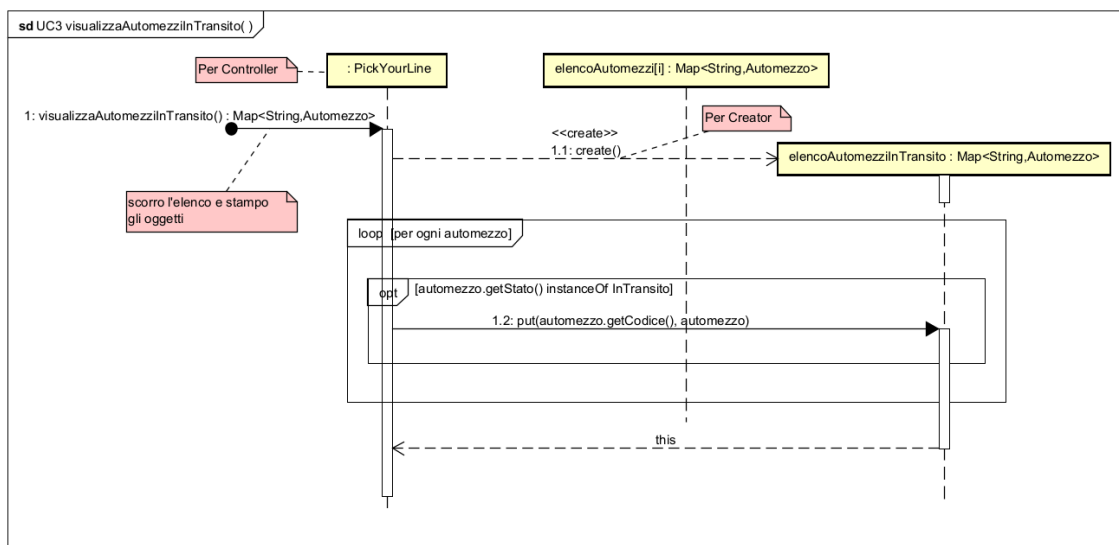
Durante questa iterazione si è ritenuto necessario eliminare la gestione degli utenti in 3 strutture diverse, in modo da avere una struttura unica per gli utenti.

Con questa modifica è possibile gestire meglio il flusso nelle operazioni di login e registrazione.

Alla luce di questo cambiamento il caso d'uso UC3 viene modificato in quanto sarebbe stato troppo oneroso valutare tutta la struttura degli utenti e inoltre fare controlli sul tipo di istanza solamente per visualizzare gli automezzi in transito (prima era una scelta lecita in quanto la struttura dei controllori era di piccole dimensioni).

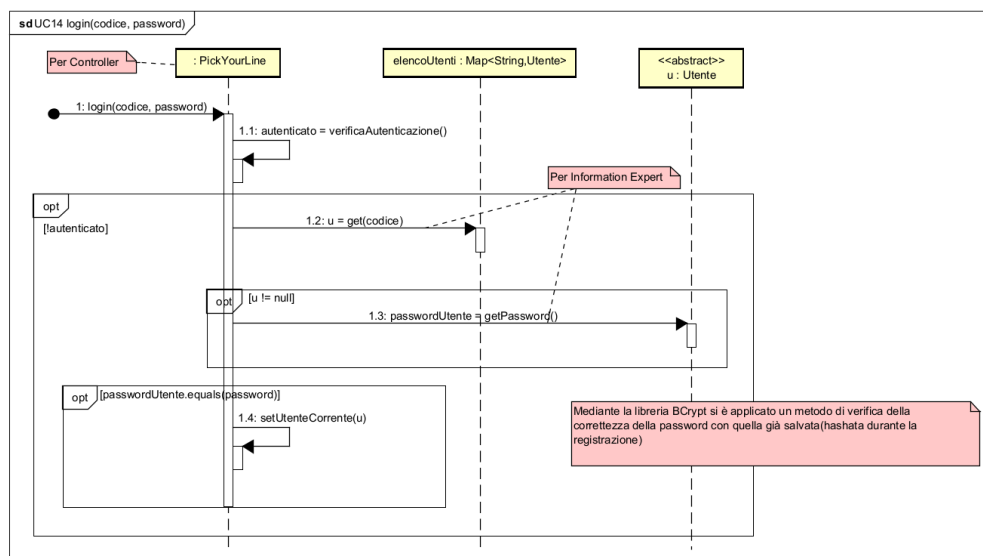
UC3

- visualizzaAutomezziInTransito



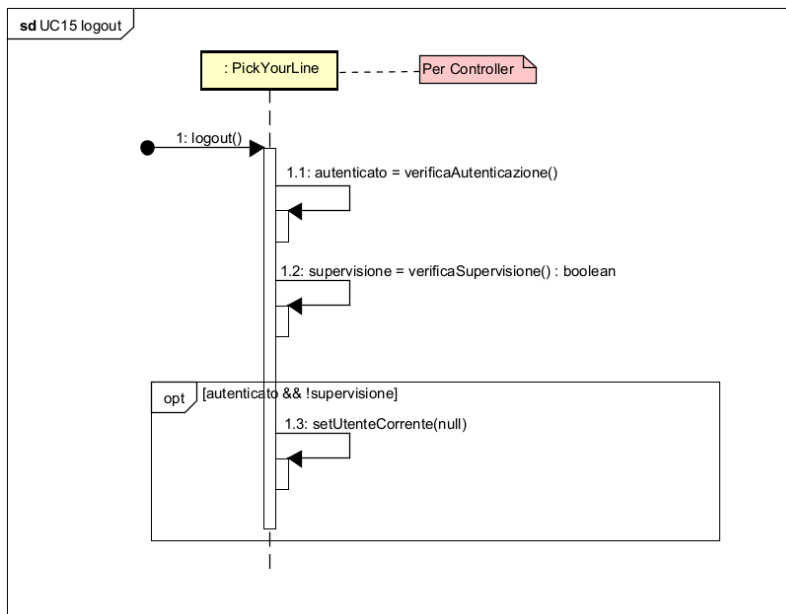
UC14

- login



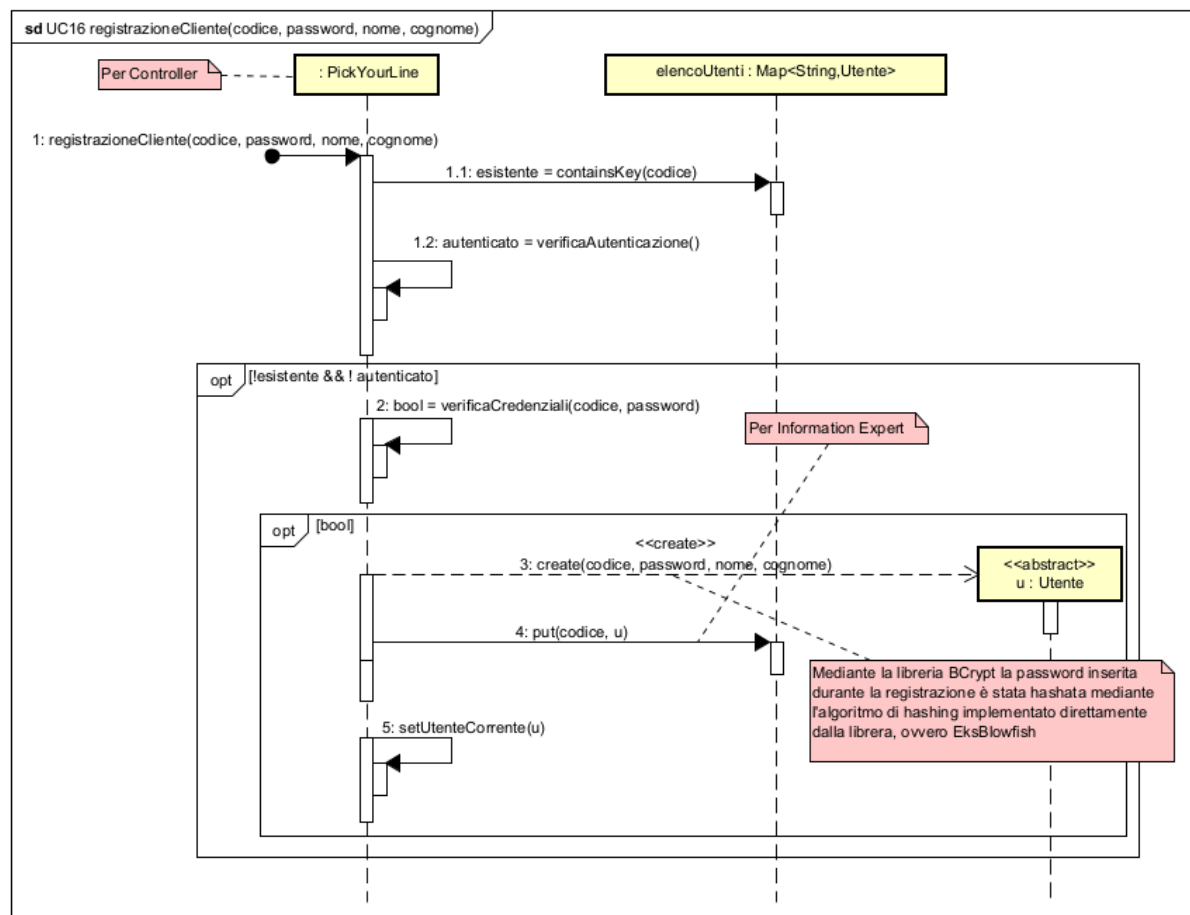
UC15

• logout



UC16

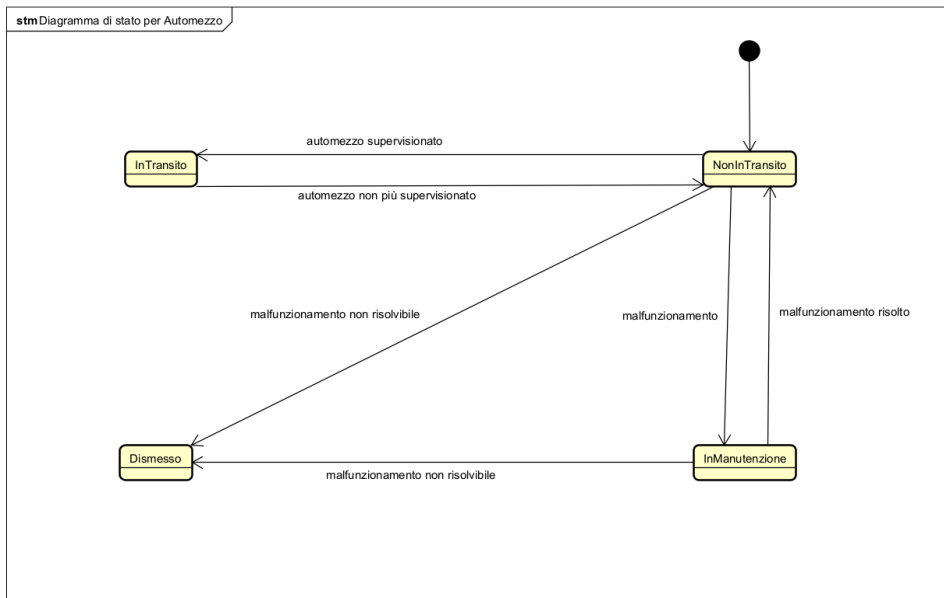
• registrazioneCliente



Diagrammi di stato

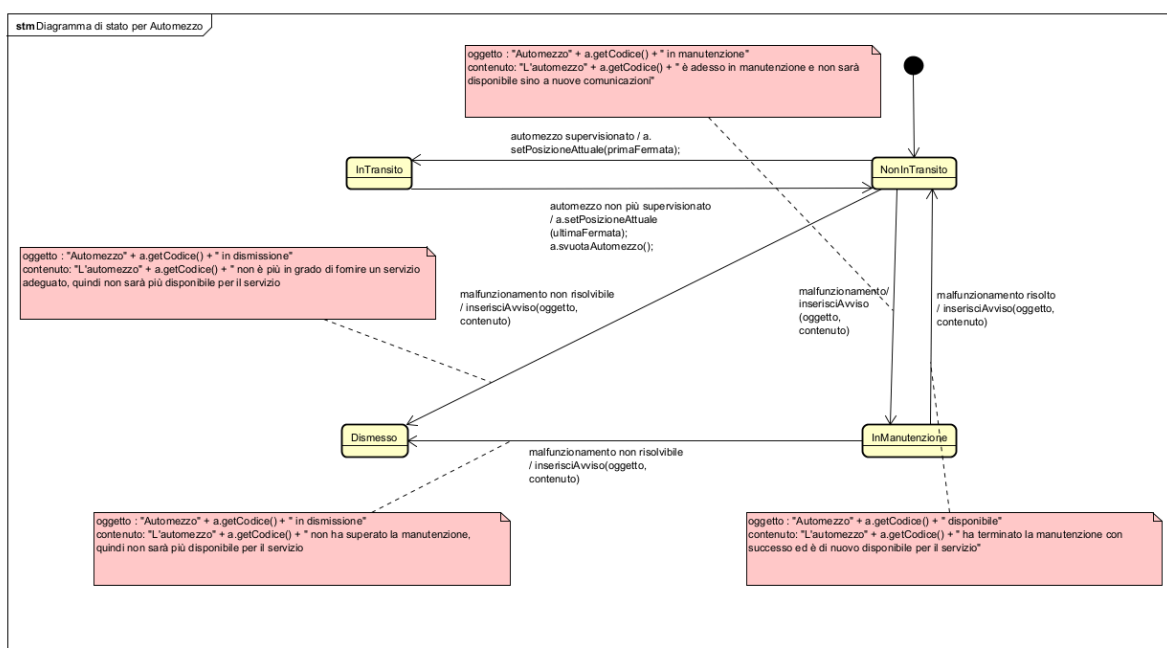
Iterazione 2

Durante questa iterazione si è pensato di implementare il pattern State solo per gestire le operazioni sugli automezzi.



Iterazione 4

Durante questa iterazione si sono aggiunte le operazioni da eseguire quando si hanno le transizioni di stato.



Testing

Introduzione

Il collaudo del software è una fase fondamentale per garantire che un'applicazione sia stabile, efficiente e conforme ai requisiti richiesti. Oltre a ridurre i costi di manutenzione, un processo di testing ben organizzato permette di individuare eventuali anomalie prima che il software venga rilasciato.

La probabilità di scoprire errori aumenta con l'aumento dei test eseguiti, ma verificare in maniera esaustiva tutte le possibili combinazioni di input risulta spesso impraticabile, a causa della complessità e del tempo richiesto. Nonostante ciò, una strategia di testing efficace aiuta a rilevare e correggere comportamenti indesiderati, migliorando l'affidabilità complessiva del sistema.

I test possono essere classificati principalmente in due categorie:

- **Test unitari:** servono a controllare il corretto funzionamento delle singole unità di codice, confrontando i risultati ottenuti con quelli attesi. Per l'automazione di questa fase, in Java si utilizza frequentemente il framework JUnit.
- **Test funzionali:** valutano il comportamento dell'intero sistema, trattandolo come una "scatola nera" e verificando che gli output generati corrispondano agli input previsti.

Nel contesto dell'applicazione sviluppata, si è scelto di concentrarsi sui test unitari, sfruttando il framework JUnit. Questo approccio, di tipo Bottom-Up, consente di validare inizialmente i singoli componenti, per poi integrarli progressivamente e verificarne il corretto funzionamento nel sistema completo.

Individuazione dei casi di test e testing unitario

Iterazione 1:

Abbiamo progettato e implementato i diversi casi di test per verificare la corretta funzionalità del caso d'uso UC1.

A seguire i test in dettaglio:

testInserisciCittaDiPartenza()

- **Scopo:** Verificare che inserendo il codice di una città di partenza, venga mostrato l'elenco delle destinazioni corretto
- **Realizzazione:** Creiamo una mappa di città attese e la confrontiamo con il risultato dell'operazione, data una specifica città di partenza

testInserisciCittaDestinazione_Successo()

- Scopo: Verificare che inserendo il codice di una città di destinazione, venga restituito il corretto elenco di itinerari disponibili
- Realizzazione: Inseriamo nell'elenco itinerari degli itinerari, verifichiamo che gli itinerari effettivamente disponibili siano stati aggiunti all'elenco

testInserisciCittaDestinazione_StessaCitta()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui città di destinazione e di partenza siano uguali
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testInserisciCittaDestinazione_CodiceNonValido()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisce un codice di città destinazione non valido
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testInserisciCittaDiPartenzaInesistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisce un codice di città partenza non esistente
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testVisualizzaItinerarioValido()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui l'itinerario non sia presente tra gli itinerari validi
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testGetSeDisponibile_Positivo()

- Scopo: Verificare se l'itinerario è disponibile data una città di partenza.
- Realizzazione: Viene settata la città corrente, viene verificato il metodo con una città di destinazione differente.

testGetSeDisponibile_Negativo()

- Scopo: Verificare se l'itinerario è disponibile data una città di partenza per percorsi inversi
- Realizzazione: Viene settata la città corrente, viene verificato il metodo con una città di destinazione differente.

testGetSeDisponibile_CittaNonNelPercorso()

- Scopo: Verificare se la città è disponibile in un itinerario nel caso in cui non è presente nel percorso
- Realizzazione: viene creata una città, viene verificato il metodo data codesta città.

testGetDestinazioniDisponibili_ContienePiuDestinazioni()

- Scopo: Verificare che l'elenco delle destinazioni disponibili contenga più di una destinazione
- Realizzazione: viene settata la città di partenza, viene verificato il metodo e valutata la dimensione del risultato e se codesto contenga le città attese

testGetDestinazioniDisponibili_ContieneUnaDestinazioni()

- Scopo: Verificare che l'elenco delle destinazioni disponibili contenga una sola destinazione
- Realizzazione: viene settata la città di partenza, viene verificato il metodo e valutata la dimensione del risultato e se codesto contenga le città attese

testGetDestinazioniDisponibili_ContieneNessunaDestinazione()

- Scopo: Verificare che l'elenco delle destinazioni disponibili non contenga nessuna destinazione
- Realizzazione: viene settata la città di partenza, viene verificato il metodo e valutata la dimensione del risultato e se codesto contenga le città attese

testGetDestinazioniDisponibili_CittaNonInPercorso()

- Scopo: Verificare che data una città, essa non appartenendo al percorso, viene restituita un elenco di destinazioni vuoto
- Realizzazione: Viene settata una città di partenza, viene verificato il metodo e valutato se il risultato sia una struttura dati vuota

Iterazione 2:

Abbiamo progettato e implementato i diversi casi di test per verificare la corretta funzionalità dei casi d'uso UC2 e UC3.

A seguire i test in dettaglio:

testTimbrabigliettoPostiConSuccesso()

- Scopo: Inserire biglietti all'interno di elencoBiglietti dell'automezzo corrente in modo corretto
- Realizzazione: Creazione di un automezzo, il relativo controllore e un elenco di biglietti, viene inserito un biglietto e viene controllato che questo venga inserito correttamente

testTimbrabigliettoPostiNonDisponibili()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si vuole timbrare un biglietto ma non ci sono più posti disponibili
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testTimbrabigliettoCittaPartenzaDiversaDaCittaAttuale()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui città di partenza sia diversa dalla città relativa alla posizione attuale

- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testTimbrabigliettoCittaCittaNonItinerario()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui la città di destinazione non sia presente nell'itinerario percorso dall'automezzo
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testConfermaInserimentoCambiaStatoBigliettoCorrente()

- Scopo: Verificare che il biglietto corrente venga inserito correttamente nell'elenco dei biglietti dell'automezzo
- Realizzazione: Viene creato un automezzo, il relativo controllore e un elenco di biglietti, viene creato un biglietto che viene settato come corrente e viene verificato il suo inserimento

testAggiornaPosizioneAutomezzoFermataNonTrovata()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui la fermata non faccia parte del percorso
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testAggiornaPosizioneAutomezzoFermataTrovata()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui la fermata venga trovata e non sia appartenente alla città di destinazione
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testAggiornaElencoBiglietti()

- Scopo: Verificare che quando viene aggiornata la posizione, i biglietti con destinazione di arrivo corrispondente alla posizione vengano rimossi dall'elenco dei biglietti.
- Realizzazione: Viene creato un automezzo a cui viene associata una lista di biglietti, viene aggiornata la posizione, e viene controllata l'effettiva rimozione dei biglietti.

testConsentiTimbratura()

- Scopo: Verificare che una volta aggiornata la posizione alla penultima città del percorso, non sia più possibile effettuare l'operazione timbraBiglietto()
- Realizzazione: Viene creato un automezzo a cui associamo un elenco biglietti, viene aggiornata la posizione alla penultima città dell'itinerario e viene verificato il valore di ritorno della funzione

testVisualizzaAutomezziInTransito()

- Scopo: Verificare che l'elenco di automezzi in transito sia restituito correttamente
- Realizzazione: Viene settato un automezzo e il relativo controllore, l'automezzo viene inserito nell'elenco degli automezzi e quest'ultimo viene confrontato con un elenco atteso

testVisualizzaAutomezzo()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui il codice dell'automezzo non esista o non sia in transito
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

Iterazione 3:

Abbiamo progettato e implementato i diversi casi di test per verificare la corretta funzionalità dei casi d'uso UC4, UC5, UC6, e UC9.

A seguire i test in dettaglio:

`testInseriscItinerario_Successo()`

- Scopo: Inserire un itinerario con successo all'interno dell'elenco degli itinerari
- Realizzazione: Vengono passati dei parametri al costruttore che non generano eccezioni e viene controllato che l'itinerario appena creato sia presente nell'elenco

`testInseriscItinerario_CodicItinerarioEsistente()`

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisca un codice appartenente a un itinerario già esistente
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

`testInseriscItinerario_OrariNonValidi()`

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inseriscano degli orari non validi (es. stesso orario di partenza e arrivo, orario arrivo precedente a orario partenza)
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

`testInseriscItinerario_PercorsoTropoCorto()`

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisce un percorso con meno di due città
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

`testInseriscItinerario_CittaDuplicateNelPercorso()`

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisce un percorso che contiene più volte la stessa città
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

`testModificItinerario_Successo()`

- Scopo: Modificare un itinerario con successo all'interno dell'elenco degli itinerari
- Realizzazione: Vengono passati dei parametri al costruttore che non generano eccezioni e viene controllato che l'itinerario appena modificato abbia i valori degli attributi diversi da quelli precedenti alla modifica

testModificaItinerario_CodiceItinerarioNonEsistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisca un codice itinerario non esistente
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testModificaItinerario_PercorsoTropoCorto()

- Scopo: Verifica che venga sollevata un'eccezione in quanto il percorso dell'itinerario possieda meno di 2 città
- Realizzazione: Creo una mappa percorso e via aggiungo una città, verifico che venga lanciata un'eccezione richiamando il metodo in test e che il messaggio di errore corrisponda a quello previsto

testModificaItinerario_CittaDuplicateNelPercorso()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui il percorso abbiamo città duplicate
- Realizzazione: Viene creato un percorso con delle città duplicate all'interno e verifico che venga lanciata un'eccezione richiamando il metodo in test e che il messaggio di errore corrisponda a quello previsto

testEliminaItinerario_ItinerarioNonEsistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia modificare un itinerario non esistente.
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testInserimentoAutomezzoValido()

- Scopo: Creare un automezzo e verificare che esso sia stato correttamente inserito all'interno dell'elenco degli automezzi.
- Realizzazione: Viene creato un automezzo e viene verificata l'esistenza di questo automezzo dentro l'elenco degli automezzi

testInserimentoAutomezzoCodiceEsistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia inserire un automezzo con codice già esistente.
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testInserimentoAutomezzoItinerarioNonEsistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia inserire un automezzo il cui l'itinerario non sia presente dentro l'elenco degli itinerari
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testModificaAutomezzoNonEsistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia modificare un automezzo non esistente

- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testModificaAutomezzo_AutomezziNonModificabili()

- Scopo: Verificare che venga lanciata l'eccezione nel momento in cui non sia possibile modificare gli automezzi
- Realizzazione: Vengono posti gli automezzi nello stato inTransito e verifico che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testModificaAutomezzoltinerario()

- Scopo: Verificare che il metodo modifichi l'itinerario dell'automezzo con successo
- Realizzazione: Inserisco un nuovo automezzo e modifico il medesimo, salvo l'automezzo modificato in una variabile ausiliaria e verifico se gli itinerari sono cambiati con successo

testEliminazioneAutomezzoEsistente()

- Scopo: Verifico che il metodo di eliminazione di un automezzo venga correttamente eseguito
- Realizzazione: Inserisco un nuovo automezzo, verifico che sia stato aggiunto all'elenco degli automezzi, elimino l'automezzo creato e verifico che sia ancora presente nell'elenco degli automezzi

testEliminazioneAutomezzoNonEsistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia eliminare un automezzo inesistente
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testVisualizzaDettaglioSegnalazioneConCodiceValido()

- Scopo: Verificare che non venga sollevata un'eccezione nel caso in cui visualizzo la segnalazione in dettaglio con codice immesso valido
- Realizzazione: Viene verificata che l'eccezione non sia sollevata

testVisualizzaDettaglioSegnalazioneConCodiceNonValido()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia visualizzare in dettaglio una segnalazione con codice immesso non valido
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testVisualizzaElencoFermateCittaNonValida()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia visualizzare l'elenco di città dato un codice di città non è esistente

- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testCreaSegnalazioneCorretto()

- Scopo: Viene verificato il corretto funzionamento durante la creazione di una segnalazione da parte del Cliente
- Realizzazione: Viene creata una segnalazione, viene applicato il metodo da verificare e confronto oggetto e contenuto della segnalazione creata con quella in uscita dal metodo testato

testInvioSegnalazione()

- Scopo: Verificare che venga inviata correttamente la segnalazione all'Amministratore
- Realizzazione: Vado a salvare l'elenco segnalazioni attuali(pre-invio), creo un cliente e lo setto come utente corrente, creo una segnalazione e applico il metodo che sto testando, verifico che l'elenco delle segnalazioni contenga la segnalazione che ho appena inviato

Iterazione 4:

Abbiamo progettato e implementato i diversi casi di test per verificare la corretta funzionalità dei casi d'uso UC7, UC8, UC11, UC12 e UC13 e raffinamento i casi d'uso UC2, UC4, UC5(solo il UC5 ha subito aggiunte di test).

A seguire i test in dettaglio:

testModificaAutomezzo_Successo()

- Scopo: Verificare che il metodo modifichi correttamente l'itinerario di un automezzo
- Realizzazione: Viene creato un nuovo automezzo e gli viene assegnato un itinerario. Subito dopo viene modificato l'automezzo e viene controllato che il nuovo itinerario sia presente dentro automezzo

testModificaAutomezzo_Stato_Successo()

- Scopo: Verificare che il metodo modifichi lo stato dell'automezzo
- Realizzazione: Si crea un automezzo e gli si assegna un itinerario. Viene cambiato l'itinerario e si controlla che il nuovo stato dell'automezzo sia In Manutenzione. Successivamente viene modificato nuovamente l'automezzo e viene controllato che il nuovo stato dell'automezzo sia NonInTransito. Successivamente viene richiamato nuovamente il metodo e viene verificato che il nuovo stato dell'automezzo sia Dismesso.

testModificaAutomezzo_AutomezziNonModificabili()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui si vuole modificare un automezzo che non può essere modificato perché in transito
- Realizzazione: Vengono posti tutti gli automezzi inTransito e viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto.

testInserisciAvviso_Successo()

- Scopo: Verificare l'inserimento di un nuovo avviso nella lista degli avvisi

- Realizzazione: Viene memorizzato il numero di avvisi, viene inserito un nuovo avviso e viene verificato che la size dell'elenco degli avvisi è più grande di 1 rispetto alla vecchia size

testModificaAvviso_Successo()

- Scopo: Verificare che il metodo modifichi l'avviso con successo
- Realizzazione: Viene recuperato un avviso dell'elenco degli avvisi e salvo il vecchio contenuto. Modifico il contenuto dell'avviso e controllo che l'oggetto sia rimasto quello vecchio mentre il contenuto sia cambiato

testModificaAvviso_AvvisoNonEsistente()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui si vuole modificare un avviso che non esiste
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testEliminaAvviso_Successo()

- Scopo: Verificare che il metodo di eliminazione venga eseguito correttamente
- Realizzazione: Viene memorizzato il numero totale di avvisi presente nell'elenco degli avvisi. Viene eliminato un controllore esistente e viene verificato che la lista degli avvisi abbia la size più piccola di 1 rispetto alla vecchia size

testEliminaAvviso_AvvisoNonEsistente()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui si vuole eliminare un avviso non esistente
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testVisualizzaElencoAvvisi_ElencoVuoto()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui l'elenco degli avvisi è vuoto
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testVisualizzaDettaglioAvviso_AvvisoNonEsistente()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui si voglia visualizzare un avviso in dettaglio il cui però codice non esiste
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testInserisciControllore_successo()

- Scopo: Inserire un controllore con successo all'interno della lista dei Controllori
- Realizzazione: Viene creato un controllore e quest'ultimo viene aggiunto all'elenco dei controllori.

testInserisciControllore_duplicato()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia inserire un controllore con codice già presente nell'elenco dei controllori
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testEliminaControllore_successo()

- Scopo: Verificare che il metodo di eliminazione venga correttamente eseguito
- Realizzazione: Viene eliminato un controllore presente nella lista dei controllori e si verifica che il codice del controllore appena eliminato non compaia nella lista dei controllori

testEliminaControllore_nonEsistente()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si voglia eliminare un controllore non esistente
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

-

testEliminaControllore_conAutomezzoSupervisionato()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui si voglia eliminare un controllore che sta supervisionando un autobus.
- Realizzazione: Prendo un controllore dall'elenco dei controllori, gli assegno un autobus e chiamo il metodo di eliminazione. Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testVisualizzaElencoAutomezziNonInTransitoElencoVuoto()

- Scopo: Verifica che venga sollevata un'eccezione se l'elenco degli automezzi disponibili è vuoto
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testSupervisionaAutomezzoConSuccesso()

- Scopo: Verifica che l'operazione venga correttamente eseguita, senza lanciare nessuna eccezione
- Realizzazione: Verifico che non venga lanciata un'eccezione all'esecuzione del metodo, verifico che l'automezzo che si intende revisionare sia coincidente con quello previsto, verifico che l'automezzo si trovi nello stato InTransito

testSupervisionaAutomezzoCodiceNonValido()

- Scopo: Verifica che venga sollevata un'eccezione se il codice dell'automezzo inserito non è valido
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testSupervisionaAutomezzoltinerarioNonAssegnato()

- Scopo: Verifica che venga sollevata un'eccezione se l'automezzo che si intende supervisionare è sprovvisto di itinerario
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testFineCorsaConSuccesso()

- Scopo: Verifica che l'operazione venga correttamente eseguita, senza lanciare nessuna eccezione
- Realizzazione: Verifico che non venga lanciata un'eccezione all'esecuzione del metodo, verifico che il controllore non supervisioni un automezzo, verifico che l'automezzo si trovi nello stato NonInTransito

testFineCorsaControlloreNonSupervisionaAutomezzo()

- Scopo: Verifica che venga sollevata un'eccezione se il controllore non supervisiona un automezzo
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

Iterazione 5:

Abbiamo progettato e implementato i diversi casi di test per verificare la corretta funzionalità dei casi d'uso UC14, UC15, UC16.

A seguire i test in dettaglio:

testLoginConSuccesso()

- Scopo: Verificare che l'operazione di login venga eseguita, senza lanciare nessuna eccezione
- Realizzazione: Viene selezionato un utente già esistente e viene eseguito il metodo da testare, osservando che non lanci nessuna eccezione

testLoginConAccessoEffettuato()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui si voglia eseguire il metodo di login con già un cliente autenticato
- Realizzazione: Viene impostato un utente come utente corrente, viene creato un altro utente e viene chiamato il metodo. Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testLoginConCodiceUtenteNonEsistente()

- Scopo: Verificare che venga sollevata l'eccezione nel caso in cui si voglia inserire un utente che non è presente nel sistema
- Realizzazione: Vengono impostati codice e password di un utente non presente nel sistema. Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testLoginConPasswordErrata()

- Scopo: Verificare che venga sollevata un'eccezione nel caso in cui si inserisca la password sbagliata per un determinato cliente.
- Realizzazione: Viene impostato il codice di un cliente presente nel sistema. Viene impostata una password per quel cliente che però è differente dalla password che il sistema ha in memoria. Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testLogout_Successo()

- Scopo: Verificare che l'operazione di logout venga eseguita con successo senza lanciare eccezioni
- Realizzazione: Viene settato un utente corrente e viene chiamato il metodo che si sta testando osservando che esso non lancia eccezioni

testLogout_AccessoNonEffettuato()

- Scopo: Verificare che venga lanciata l'eccezione nel caso in cui si voglia effettuare il logout senza però prima aver effettuato l'accesso
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testLogout_SupervisioneInCorso()

- Scopo: Viene verificato che venga sollevata l'eccezione nel caso in cui l'utente stia supervisionando un autobus
- Realizzazione: Viene settato come utente corrente un controllore che sta supervisionando un automezzo. Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testRegistrazioneCliente_ConSuccesso()

- Scopo: Verificare il corretto funzionamento dell'operazione "Registrazione Cliente"
- Realizzazione: Settato l'utenteCorrente a nulla; lanciato l'operazione e verificato se l'elenco degli utenti contenesse il codice dell'utente che si è appena registrato

testRegistrazioneCliente_ClienteAutenticato()

- Scopo: Verificare che venga lanciata l'eccezione durante l'operazione "Registrazione Cliente" nel momento in cui il Cliente è già autenticato
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testRegistrazioneCliente_ClienteGiaRegistrato()

- Scopo: Verificare che venga lanciata l'eccezione durante l'operazione "Registrazione Cliente" nel momento in cui il Cliente è già registrato
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

testRegistrazioneCliente_PasswordNonValida()

- Scopo: Verificare che venga lanciata l'eccezione nel caso in cui si voglia registrare il cliente avente una password non corrispondente ai criteri imposti dal sistema
- Realizzazione: Viene verificata che l'eccezione sia sollevata e che il messaggio corrisponda a quello previsto

Test di sistema

Una fase di test manuale per effettuare una verifica del complessivo comportamento del software, in un ambiente più vicino a quello reale.

Verifica dell'interfaccia utente:

Obiettivo: Garantire che l'interfaccia sia intuitiva, nel nostro caso non implementando un'interfaccia grafica vera e propria quantomeno dare una chiara visione di ciò l'applicazione offre come funzionalità, rispondendo alle richieste finali dell'utente finale(Cliente,Controllore,Amministratore)

Attività: Navigazione nel menù principale per valutare la chiarezza e ordine della struttura

Test delle funzionalità principali:

Obiettivo: Verificare che tutte le funzionalità offerte dell'applicazione fossero correttamente implementate e che rispondessero ai requisiti

Attività: Abbiamo simulato degli scenari reali in cui l'utente si potesse ritrovare durante l'uso dell'applicazione: ad esempio abbiamo verificato l'operazione "Invio Segnalazione" in caso in cui l'utente autenticato mandasse una segnalazione agli Amministratori del servizio, nel caso in cui l'Utente è un Amministratore, la possibilità di aggiungere nuovi Controllori nel sistema affinché possano lavorare e garantire supporto al servizio di trasporto.

Analisi della Coerenza dei Dati:

Obiettivo: Assicurarsi che i dati visualizzati fossero coerenti con quelli memorizzati nel sistema.

Attività: Abbiamo verificato la consistenza e correttezza dei risultati visualizzati a schermo dalle operazioni come Cercaltinerario o MonitoraAutomezzo; da Amministratore abbiamo creato Automezzi e visualizzati eventualmente se l'automezzo creato risultasse presente durante il VisualizzaAutomezzi.

Valutazione delle Prestazioni:

Obiettivo: Garantire che l'applicazione fosse reattiva e performante durante le interazioni utente senza rallentamenti o interruzioni impreviste.

Attività: Durante la simulazione di uno scenario, abbiamo previsto già dei dati inseriti dentro il sistema: come ad esempio Controllori, Segnalazioni o Itinerari. Inoltre durante il processo di RegistrazioneCliente e InserisciControllore è previsto l'hashing della password: processo effettuato sul momento, visto che attuarlo da costruttore, anche se intuitivo, porterebbe ad un sovraccarico del sistema con conseguente rallentamento durante l'avviamento e test.

In conclusione tutti i test effettuati hanno offerto risultati positivi, confermando che l'applicazione rispettasse tutti i requisiti prefissati e che le operazioni fossero implementati nella maniera desiderata.